



Data Management

Aimée Lynn Backiel
Wim Bertels
Alain Van Der Jeught

Module B. Advanced SQL Transactions

TI / BCS Attribution-NonCommercial-ShareAlike 4.0 Unported Licence 2025-2026

1

SYMBOLS USED IN THIS COURSE

Symbol	Category	Typical Slide Use
	Positioning	Positioning, purpose
	Objective	Objective, goal
	Attention	Warnings, key pitfalls, critical notes
	Theory	Concept explanations, definitions
	Practice	Exercises, hands-on labs
	Clip	Demo, Clip
	Poll	Live questions / Quiz
	Informational	Neutral notes, references, reading tips. Not examinable

2

PURPOSE

- Ensure **data integrity** — all related changes succeed or none do.
- Protect against **partial updates** in case of errors or failures.
- Control **concurrent access** so multiple users can work without corrupting data.
- Support **complex operations** spanning multiple SQL statements.
- Enable **safe rollback** to a consistent state if something goes wrong.

3

OBJECTIVES

- **Define** what a transaction is and explain its ACID properties.
- **Start, commit, and roll back** transactions using SQL commands.
- **Use savepoints** to roll back part of a transaction.

4

DEFINITION

- A **transaction** is a sequence of one or more SQL statements that are executed as a single unit of work.
- The transaction must be **fully completed** (committed) so all changes become permanent, or **fully undone** (rolled back) so the database returns to its original state, ensuring data integrity.
- **ACID** describes the four main promises a database transaction makes to keep data safe and reliable.

5

ACID PROPERTIES

- **A — Atomicity (“All or Nothing”)**
- A transaction’s operations are **all completed successfully or none at all**.
- If one part fails, the entire transaction is **rolled back**.
- Example: In a bank transfer, money is **deducted from one account and added to another** — never just one of them.

6

ACID PROPERTIES

- **C — Consistency (“Valid State”)**
- A transaction brings the database from one **valid state** to another valid state, following all **rules, constraints, and triggers**.
- No transaction can break **data integrity** rules (like unique constraints or foreign keys).
- Example: You can’t insert a row with an invalid foreign key.

Slide 7

7

ACID PROPERTIES

- **I — Isolation (“No Interference”)**
- The operations of one transaction are **invisible** to others until the transaction is committed.
- Prevents concurrency problems like **dirty reads** or **lost updates**.
- Example: You can’t see a friend’s half-completed edit in an online order system.

Slide 8

8

ACID PROPERTIES

- **D — Durability (“It Sticks”)**
- Once a transaction is **committed**, the changes are **permanent** — even if the system crashes.
- The database stores committed changes in **non-volatile memory** (disk, transaction logs).
- Example: After hitting “Place Order,” your order remains recorded even if the server restarts.

Slide 9

9

COMMIT AND ROLLBACK

- **Commit:** making a transaction permanent
- **Rollback:** undoing a transaction
 - Once committed, no rollback is possible!
- **Autocommit:**
 - Every SQL statement is a transaction
 - Every transaction commits automatically if it succeeds
 - Errors roll back that single SQL statement
 - Autocommit is **default behaviour in PostgreSQL**

Slide 10

10

TRANSACTION PROCESSING

- **Implicit start**
 - A new transaction begins automatically after a COMMIT or ROLLBACK when autocommit mode is enabled.
- **Explicit start**
 - **BEGIN;** sql code; **COMMIT;** -- dialect
 - **BEGIN;** sql code; **ROLLBACK;** -- dialect
 - **START TRANSACTION;** sql code; **COMMIT;**
 - **START TRANSACTION;** sql code; **ROLLBACK;**
 - Always **end** a transaction explicitly — don’t leave it open.

Slide 11

11

TRANSACTION PROCESSING

- When does it make sense ?
 - If a particular entry is to be deleted from multiple tables
 - If a user made a mistake in adjustments
 - ..
- Possible exceptions (product restrictions) : Some commands, like catalog modifications (e.g., CREATE TABLE, ALTER TABLE), may be handled outside normal transaction control.

Slide 12

12

SAVEPOINTS

- Purpose:
 - Allow you to **undo part of the current transaction** without rolling back the entire transaction.
 - Act as **intermediate checkpoint (snapshot)** within a transaction.
- You can have multiple savepoints in a single transaction.
- Rolling back to a savepoint **does not end the transaction**.

Slide 13

13

SAVEPOINTS

- Example script template:


```
START TRANSACTION;
UPDATE ...;
INSERT ...;
SAVEPOINT S1; -- first checkpoint
INSERT ...
SAVEPOINT S2; -- second checkpoint
DELETE ...;
ROLLBACK TO SAVEPOINT S2; -- undo changes after S2 only
...
```

Slide 14

14

SAVEPOINT EXAMPLE SCRIPT

```
START TRANSACTION;
-- Step 1: Add a new star
INSERT INTO celestial_object (objectname, satellite_of, diameter, distance)
VALUES ('Alpha Cent', NULL, 1200000, 41300000);
-- Step 2: Add an exoplanet orbiting the new star
INSERT INTO celestial_object (objectname, satellite_of, diameter, distance)
VALUES ('Proxima b', 'Alpha Cent', 12000, 42330000);
-- Savepoint after adding both
SAVEPOINT sp1;
-- Step 3: Update exoplanet's diameter
UPDATE celestial_object
SET diameter = 12500
WHERE objectname = 'Proxima b';
-- Savepoint after update
SAVEPOINT sp2;
-- Step 4: Roll back to sp2
ROLLBACK TO SAVEPOINT sp2;
-- Step 5: Update exoplanet again
UPDATE celestial_object
SET diameter = 13000
WHERE objectname = 'Proxima b';
-- Savepoint after second update
SAVEPOINT sp3;
-- Step 6: Roll back to sp1
ROLLBACK TO SAVEPOINT sp1;
-- Step 7: Update star's distance
UPDATE celestial_object
SET distance = 4131500
WHERE objectname = 'Alpha Cent';
-- Step 8: Roll back to sp3
ROLLBACK TO SAVEPOINT sp3;
-- Commit final changes
COMMIT;
```

- **After Step 2** — What rows should now exist in celestial_object related to Alpha Cent?
- **After Step 4** — Did anything change in the table after rolling back to sp2 immediately after creating it? Why or why not?
- **After Step 5** — What is the diameter of Proxima b?
- **After Step 6 (rollback to sp1)** — What is the diameter of Proxima b now? Why?
- **After Step 7** — What has changed in Alpha Cent's distance compared to the original insert?
- **If we try to rollback to sp3 after step 7** — What happens?

Slide 15

15

OPO: Data Management

Aimée Lynn Backiel
Wim Bertels
Alain Van Der Jeught

Module B: Advanced SQL Prepared Statements

TI / BCS

Attribution-NonCommercial-ShareAlike 4.0 Unported Licence

2025-2026

16

SYMBOLS USED IN THIS COURSE

Symbol	Category	Typical Slide Use
	Positioning	Positioning, purpose
	Objective	Objective, goal
	Attention	Warnings, key pitfalls, critical notes
	Theory	Concept explanations, definitions
	Practice	Exercises, hands-on labs
	Clip	Demo, Clip
	Poll	Live questions / Quiz
	Informational	Neutral notes, references, reading tips. Not examinable

Slide 17

17

PURPOSE

- Improve **performance** by parsing, planning, and optimizing a query once, then reusing it multiple times.
- Enhance **security** by preventing SQL injection attacks (safe parameter binding).
- Support **maintainability**: SQL is separated from dynamic user input, making code cleaner and more reliable.
- Useful in both **application programming** (Java, Python, etc.) and **within PostgreSQL** (PREPARE / EXECUTE).

Slide 18

18

OBJECTIVES

- **Define** what a prepared statement is and why it is used.
- **Explain** the difference between ad-hoc SQL execution and prepared execution.
- **Create and use** prepared statements in PostgreSQL (PREPARE, EXECUTE, DEALLOCATE).
- **Apply** prepared statements in application code.
- **Evaluate** the advantages:
 - Performance gains from plan reuse.
 - Security improvements (SQL injection prevention).
 - Code readability and maintainability.

Slide 19

19

PREPARED STATEMENTS

- **What?**
 - Server-side object
 - Possible parameters
- Can be used to **optimise performance**
 - Run similar queries multiple times
 - Complex queries with many joins
- Only takes as long as the current DB connection or when DEALLOCATE is used
 - Cannot be used by multiple DB clients/sessions
 - Each client can have its own prepared statements, session dependent
- **Syntax:**
 - PREPARE name [(data_type [, ...])] AS statement

Slide 20

20

AD-HOC SQL VS PREPARED EXECUTION

- **Ad-hoc SQL Execution**
 - Each query is sent as raw SQL text to the database.
 - For every execution, the DBMS must:
 1. Parse the SQL string (syntax check).
 2. Analyse objects (tables, columns, types).
 3. Plan/optimize the query execution strategy.
 4. Execute the query.
 - If the same query runs x times with different values, steps 1–3 repeat x times.
- **Prepared Execution**
 - The SQL statement is parsed, validated, and planned once.
 - The execution plan is stored server-side in the session.
 - Subsequent runs just bind new parameter values and directly execute the cached plan.

Slide 21

21

PREPARED STATEMENTS

- **Flow**
 - PREPARE -- You define a prepared statement once
 - EXECUTE -- You run the prepared statement, passing parameter values if needed
 - ... -- You can EXECUTE the prepared statement as many times as you like during the session.
 - DEALLOCATE -- You remove the prepared statement from memory.
- **Example:**
PREPARE add_customer(numeric(4,0), varchar(20), varchar(20), date) **AS INSERT INTO** customers **VALUES**(\$1, \$2, \$3, \$4);
EXECUTE add_customer(100, 'Lieve', 'Standaard', now());

Slide 22

22

PREPARED STATEMENTS

- | | |
|--|--|
| <ul style="list-style-type: none">• PREPARE statement -> statement is...<ul style="list-style-type: none">◦ Parsed◦ Analysed◦ Rewritten• EXECUTE statement -> statement is...<ul style="list-style-type: none">◦ Planned◦ Executed | <ul style="list-style-type: none">• DEALLOCATE statement -> statement is...<ul style="list-style-type: none">◦ Dropped |
|--|--|

Slide 23

23

PREPARED STATEMENTS

- Underlying a Prepared Statement can be
 - SELECT, UPDATE, DELETE, INSERT, ...
 - With parameters substituted according to position, use \$1, \$2, etc.
 - Write security: Advantageous because no other queries can be executed
 - A prepared statement will execute what was prepared
- List of current prepared statements in your session (PostgreSQL):
SELECT *
FROM pg_prepared_statements;

Slide 24

24

EVALUATING THE ADVANTAGES

- **Performance**
 - Query is **parsed, planned, and optimized once**.
 - Execution plan is **reused** across multiple runs.
 - Significant gain for **frequently executed or complex queries** (many joins, aggregations).
- **Security**
 - User input is passed as **parameters**, not concatenated strings.
 - Prevents **SQL injection attacks** by separating SQL code from data.
- **Maintainability**
 - SQL text is **cleaner**: placeholders instead of inline values.
 - Code is easier to **read and maintain** in applications.
 - Encourages consistent, reusable query definitions.

Slide 25

25

PREPARE SYNTAX

- **PREPARE** name [(parameter [, ...])] AS ..
- Example:
PREPARE add_customer(numeric(4,0), varchar(20), varchar(20), date) **AS** INSERT INTO customers VALUES(\$1, \$2, \$3, \$4);

Slide 26

26

EXECUTE SYNTAX

- **EXECUTE** name [(parameter [, ...])]
- Example:
EXECUTE add_customer(100, 'Lieve', 'Standaert', now());
- Note
 - A list of parameter data types can also be specified, e.g. **EXECUTE** add_customer(100, 'Lieve', 'Standaert', now()::timestamp); -- :: is a dialect for cast
 - When the data type of a parameter is unknown, the data type is derived from the context where the parameter was used (if possible).

Slide 27

27

DEALLOCATE SYNTAX

- **DEALLOCATE** { name | ALL }
- Example:
DEALLOCATE add_customer;
- **DEALLOCATE** ALL;

Slide 28

28

EMBEDDED PREPARED STATEMENTS

- Embedded in another language, e.g. in Java:
- Prepared Statement add_customer =
c.prepareStatement("INSERT INTO customers
VALUES(?, ?, ?, ?)");

Slide 29

29

ISO?

- Context:
- The SQL standard includes a PREPARE statement, but this is only intended for embedded SQL.
- The direct PostgreSQL PREPARE statement uses slightly different syntax from this standard, it is an addition.
- More info:
<https://www.postgresql.org/docs/current/sql-prepare.html>

Slide 30

30



OPO: Data Management

Aimée Lynn Backiel
Wim Bertels
Alain Van Der Jeught

Module B: Advanced SQL Procedural SQL

TI / BCS Attribution-NonCommercial-ShareAlike 4.0 Unported Licence 2025-2026

31

SYMBOLS USED IN THIS COURSE

Symbol	Category	Typical Slide Use
	Positioning	Positioning, purpose
	Objective	Objective, goal
	Attention	Warnings, key pitfalls, critical notes
	Theory	Concept explanations, definitions
	Practice	Exercises, hands-on labs
	Clip	Demo, Clip
	Poll	Live questions / Quiz
	Informational	Neutral notes, references, reading tips. Not examinable

32

PURPOSE

- To extend **standard SQL** with procedural constructs (loops, conditions, variables).
- To allow **complex business logic** to run directly inside the database.
- To improve **performance** by reducing round-trips between application and DB.
- To enable **modularity** with stored procedures, functions, and triggers.
- To provide **control flow** not available in plain SQL.

33

OBJECTIVES

- Explain the difference** between declarative SQL and procedural SQL.
- Define and use procedural constructs** such as variables, loops, and conditions.
- Write stored procedures, functions and triggers** in a procedural SQL language (e.g., PL/pgSQL in PostgreSQL).
- Implement business logic** inside the database using procedural SQL.
- Compare use cases:** when to use procedural SQL vs keeping logic in the application layer.

34

INTRODUCTION TO PROCEDURAL SQL

- SQL/PSM** (Persistent Stored Modules): defines how to write **stored procedures, functions, local variables, control flow (IF, WHILE, FOR), exception handling**, etc
- ISO Standard since 1996, 1999, ..., 2023
- Many products focus on the standard, but none are following it completely.
 - Check the documentation of the concrete product.

35

DECLARATIVE VS PROCEDURAL SQL

- Declarative SQL**
 - Standard SQL
 - What: You tell the DBMS **what result you want**, not how to get it.
 - Focus: What data to retrieve/manipulate.
- Procedural SQL**
 - SQL/PSM, PL/pgSQL, etc.
 - What: You tell the DBMS **how to do things step by step**, with control flow.
 - Focus: How the operations should be executed.

36

USAGE OF SQL/PSM

- **Stored Procedures**
 - Encapsulate complex business logic (e.g., monthly billing, payroll calculation).
 - Can be executed as a single call instead of multiple SQL statements.
 - Reduce network traffic between application ↔ database to improve performance.
- **User-Defined Functions (UDFs)**
 - Return values or result sets.
 - Allow reuse of logic inside queries.
 - Example: a function to compute distance between two celestial objects.
- **Control Flow in SQL**
 - Add IF, CASE, WHILE, FOR loops directly in SQL.
 - Enables conditional logic and iterative processing.
- **Error Handling (Exception Handling)**
 - Useful for robust transaction control.
- **Triggers**
 - Although not formally part of SQL/PSM, most procedural SQL implementations allow triggers to run stored procedures automatically (on INSERT, UPDATE, DELETE).

Slide 37

37

POSTGRESQL

- Supports several programming languages:
 - SQL
 - PL/pgSQL (standard) ← Scope of this presentation
 - PL/Tcl
 - PL/Perl
 - PL/Python
 - PL/Java (extension)
 - PL/Lua (extension)
 - PL/R (extension)
 - PL/sh (extension)
 - ..

Slide 38

38

"TRUSTED VS. NON-TRUSTED"

- **Trusted:**
 - these languages are restricted to what is authorised by the database itself (DBMS)
- **Untrusted:**
 - these languages are not restricted for this purpose.
 - these allow **direct access to the operating system** (files, network, processes).
 - superuser access required to use them.

Slide 39

39

ADDITIONAL MODULES AVAILABLE

- Postgresql comes with some additional modules such as:
- Debugger (e.g. pldebugger for pgadmin)
- Profiler (e.g. plprofiler, or more generally pg_stat_statements)

Slide 40

40

OBJECTS ON THE SERVER

- In the context of **SQL/PSM**, certain **database objects** can contain executable code. These objects are stored in the **database catalog** (system tables) and live inside the DBMS.
- Types of server-side code objects
 - Stored procedures
 - Stored functions
 - Triggers
- The catalog can store executable code objects. This is the code **saved inside the DBMS** that can be **activated (executed)** when called or when certain events occur.
- Similar to (static) methods in programming languages.

Slide 41

41

EXAMPLE: STORED FUNCTION

```
CREATE OR REPLACE FUNCTION increment(i INT) -- function takes one
parameter of type integer
RETURNS INT -- function will return integer value
AS
$$ -- the function body is written between these dollar quotes
BEGIN -- start of a procedural block
RETURN i + 1; -- returns the result
END; -- end of a procedural block
$$ LANGUAGE 'plpgsql'; -- written in PL/pgSQL

-- An example of how to call the function:
SELECT increment(10);
```

Slide 42

42

PROCESSING

- A Procedure/function is **called from a program or a query**.
- **DBMS receives the call** and looks up procedure/function in its catalog.
- The **interpreter executes the stored code**, which can include SQL queries and procedural logic.
- **Execution either:**
 - **returns a result (function)**, or
 - **completes with no return value (procedure)**.
- If successful, the caller gets the result. If not, PostgreSQL raises an error/exception with a SQLSTATE code.

Slide 43

43

PARAMETERS GENERAL

- **Parameters** are how stored procedures and functions communicate with the outside world (application or calling SQL)
- 3 types (signature method) :
 - Input parameters: **IN**
 - Output parameters: **OUT**
 - Input/output parameters: **INOUT**
- Naming best practice
 - Avoid naming parameters the same as table columns (to prevent ambiguity in queries inside the function).
 - If unavoidable, use aliases (\$1, \$2, ...) or qualify table columns with table alias.
- Return values
 - **Functions must have a RETURNS** clause (scalar, record, or table).
 - **Procedures** don't return a value directly but can use OUT or INOUT parameters to pass data back.

Slide 44

44

EXAMPLE IN AND OUT

```
CREATE FUNCTION dup(IN int, OUT f1 int, OUT f2 text)
AS
$$
    SELECT $1, CAST($1 AS text) || ' is text' -- $1 is first
    parameter
$$
→ LANGUAGE SQL;

SELECT * FROM dup(42);
```

f1	f2
42	42 is text

Slide 45

45

EXAMPLE IN AND OUT

```
CREATE FUNCTION dup2(IN i int, OUT f1 int, OUT f2 text)
AS
$$
    BEGIN
        f1 := i;
        f2 := CAST(i AS text) || ' is text';
    END
$$
→ LANGUAGE plpgsql;

SELECT * FROM dup2(42);
```

functions with OUT params don't need an explicit RETURNS in the signature — it's implied from the OUT list

f1	f2
42	42 is text

Slide 46

46

SIGNATURE DIFFERENCES: FUNCTIONS VS PROCEDURES

- **Functions**
 - Always **return something**.
 - Signature includes:
 - Function name
 - Parameters (IN, OUT, INOUT)
 - **Return type (RETURNS)** — If you define **OUT** or **INOUT** parameters in a function, they **act as the return values** and **RETURNS** can be omitted.
 - Can be called inside SQL SELECT expressions
- **Procedures**
 - **Do not have a return type** (no RETURNS).
 - Signature includes only:
 - Procedure name
 - Parameters (IN, OUT, INOUT)
 - Called with **CALL**, not in a SELECT.

Slide 47

47

WHY SIGNATURE DIFFERENCES?

- Historically:
 - **functions = calculations**
 - **procedures = manipulations**
- In addition:
 - **Functions:** must return a value; **no transaction control allowed**.
 - **Procedures:** no return value; **transaction control allowed**.
- The planner optimizes functions more aggressively, while procedures are treated as standalone execution units.
 - Functions can be optimized like mathematical expressions inside a query.
 - Procedures are black boxes — the planner just executes them as they are, without optimization across boundaries.

Slide 48

48

THE 4 BUILDING BLOCKS OF PROGRAMMING LOGIC

- Declaration
- Sequence
- Selection
 - IF ... THEN ... (ELSE ...) END IF
 - CASE ... WHEN ... THEN ... END CASE
- Iteration
 - FOREACH ... LOOP ... END LOOP
 - FOR ... IN ... LOOP ... END LOOP

Apply also to procedural SQL.

Slide 49

49

STRUCTURE: FUNCTIONS

```
CREATE FUNCTION function_name(arg1,arg2,...)
RETURNS type
AS
' -- Dollar quoting ($$ ... $$) is recommended instead of single quotes (' ... ')
BEGIN
-- logic
END
'
LANGUAGE language_name;
```

Slide 50

50

STRUCTURE: FUNCTIONS

- Specify name of function
- List of parameters after function name
- Define return type
- Followed by code within BEGIN and END block
- Which procedural language

Slide 51

51

REMOVING STORED FUNCTIONS

- DROP FUNCTION: deletes function
- Example:
 - DROP FUNCTION function_name;
 - DROP FUNCTION function_name(signature);

Slide 52

52

SELECT INTO

- **SELECT INTO** assigns the **result of a query** to a **variable** inside PL/pgSQL.
- Used to fetch single values (scalars) or an entire row into a record/rowtype variable.
- Example:

```
CREATE FUNCTION sum_fines_player(IN p_player_nr INT)
RETURNS DECIMAL(8,2)
AS
$anything$
DECLARE sum_fines decimal(8,2);
BEGIN
    SELECT SUM(amount) INTO sum_fines
    FROM fine
    WHERE player_nr = p_player_nr;
    RETURN sum_fines;
END;
$anything$
LANGUAGE plpgsql;

SELECT sum_fines_player(27);
```

Slide 53

53

PERFORM

- Results of a SQL statement should be captured, e.g. via INTO variable.
- **PERFORM** is the PL/pgSQL alternative to SELECT when you don't want to capture or return the result.
 - Example
 - SELECT now(); will return an error in a code block
 - PERFORM now(); not
 - Similar to voiding a function in other programming languages

Slide 54

54

FOUND GLOBAL VARIABLE

- **FOUND** is a **global boolean variable** that is **automatically set** after certain statements in PL/pgSQL.
- Tells you if the **last SQL statement** inside the PL/pgSQL block affected or returned any rows.
- Example:

```
PERFORM player_nr FROM player ;
IF FOUND THEN ... END IF ;
```

Slide 55

55

VARIABLE DECLARATIONS IN PG/PLSQL

Declaration Form	Meaning / Usage	Example
name type;	Declare a variable with explicit SQL type.	v_dist numeric;
name type := value;	Declare with type and assign initial value.	counter int := 0;
name CONSTANT type := value;	Read-only variable (must assign value immediately).	pi CONSTANT numeric := 3.14159;
name table.col%TYPE;	Variable with same type as a table column.	v_diameter space_travel.celestial_object.diameter%TYPE;
name table%ROWTYPE;	Holds a full row (composite type) from the table.	rec space_travel.celestial_object%ROWTYPE;
name record;	Generic row-type variable (structure adapts to SELECT INTO).	rec record;
DECLARE ... CURSOR FOR <query>	Define a cursor variable for row-by-row processing.	CURSOR c_obj IS SELECT * FROM space_travel.celestial_object;

Slide 56

56

RETURNS IN SIGNATURE / RETURN IN CODE

RETURNS Signature	Meaning	Inside Code	Example
RETURNS type (scalar)	One single value	RETURN <value>;	RETURN 42;
RETURNS composite_type	One row (all columns of table/type)	RETURN <row_variable>;	RETURN r;
RETURNS SETOF type (scalar set)	Multiple values (one column)	RETURN NEXT <value>; (loop) or RETURN QUERY SELECT ...;	RETURN QUERY SELECT objectname FROM ...;
RETURNS SETOF composite_type	Multiple rows (whole table rows)	RETURN NEXT <row_variable>; (loop); or RETURN QUERY SELECT ...;	RETURN QUERY SELECT * FROM space_travel.celestial_object;
RETURNS TABLE (col1 type, ...)	Rowset with named output columns	RETURN QUERY SELECT col1, col2 ...; (or RETURN NEXT ...);	RETURN QUERY SELECT objectname, distance FROM ...;
RETURNS void	No return value	Just RETURN; (or omit)	RETURN;

Slide 57

57

RAISE

- The **RAISE** statement is used in PL/pgSQL to:
 - Send messages back to the client (for debugging or logging).
 - Report errors, warnings, or info during function execution.
 - Control exception handling (by raising exceptions you can catch in EXCEPTION blocks).
- Levels
 - **NOTICE**: Informational message (default if no level given).
 - **INFO**: More verbose info.
 - **DEBUG**: Debugging messages (visible with client_min_messages).
 - **WARNING**: Warns the user.
 - **EXCEPTION**: Stops execution and raises an error.
- Example
 - **RAISE NOTICE** 'Object % has distance %', p_name, v_dist;

Slide 58

58

RAISE USING

- **RAISE .. USING** enrich the message with details
 - **ERRCODE** = 'unique_violation',
 - **HINT** = 'suggest the reason for the user',
 - **DETAIL** = 'more detail error',
 - **MESSAGE** = 'behaviour of the unique_violation';
- Example:

```
RAISE EXCEPTION 'Distance is invalid'
USING DETAIL = 'Distance must be greater than zero',
HINT = 'Check the value passed to the function';
```

Slide 59

59

ERROR MESSAGES

- Error messages :
 - SQL error code : descriptive text
 - SQLSTATE : code (number)
- Example:

```
DO
$$
BEGIN
INSERT INTO space_travel.celestial_object(objectname)
VALUES ('Earth'); -- already exists, PK violation
EXCEPTION
WHEN unique_violation THEN
RAISE NOTICE 'Caught a duplicate key! SQLSTATE = %, SQLERRM = %',
SQLSTATE, SQLERRM;
END;
$$ LANGUAGE plpgsql;
```

=> Caught a duplicate key! SQLSTATE = 23505, SQLERRM = duplicate key value violates unique constraint "celestial_object_pkey"

DO is dialect,
<https://www.postgresql.org/docs/current/sql-do.html>

Slide 60

60

ASSERT

- ASSERT is a statement for debugging.
- Syntax: ASSERT condition [, message];
- It checks whether a given condition is true at runtime.
 - If the condition is true → execution continues normally.
 - If the condition is false → an error is raised (exception).
- Example:


```
DO
$$
DECLARE
    object_count integer;
BEGIN
    SELECT COUNT(*) INTO object_count
    FROM celestial_object;
    ASSERT object_count > 0, 'No celestial objects found, check the celestial_object table';
END;
$$
```

SHOW plpgsql.check_asserts;

Slide 61

61

SECURITY

- INVOKER : Function runs with the privileges of the **caller** (the user executing it) -> default and safe
- DEFINER : Function runs with the privileges of the **function owner** (the one who created it) -> must be specified.
- Example:


```
-- Table restricted to admins
CREATE TABLE space_travel.secret_data(id int, notes text);
INSERT INTO space_travel.secret_data VALUES (1, 'this is a secret');
-- Function allows others to view data safely
CREATE OR REPLACE FUNCTION space_travel.get_notes(OUT v_notes text)
AS
$$
BEGIN
    SELECT notes INTO v_notes FROM space_travel.secret_data;
END;
$$
LANGUAGE plpgsql
SECURITY DEFINER;
```

--run the function:
SELECT get_notes();
- non-admin users can now run SELECT * FROM space_travel.get_notes(), but they cannot query secret_data directly.

Slide 62

62

IMMUTABILITY LEVELS

The **immutability** level of a function tells the planner how the function behaves with respect to its inputs and the database state. The planner can use that info for query optimization.

Level	Meaning	Optimizer Use	Example (space_travel)
IMMUTABLE	Always returns the same result for the same input. Cannot depend on DB state or time.	Can be precomputed and used in indexes.	CREATE FUNCTION km_to_miles(d numeric) RETURNS numeric AS \$\$ SELECT d * 0.621 \$\$ LANGUAGE sql IMMUTABLE;
STABLE	Same within a query, can change between queries. May read DB, but cannot modify it.	Evaluated once per query, cached.	CREATE FUNCTION get_planet_count() RETURNS int AS \$\$ SELECT count(*) FROM space_travel.celestial_object \$\$ LANGUAGE sql STABLE;

Slide 63

63

IMMUTABILITY LEVELS

Level	Meaning	Optimizer Use	Example (space_travel)
VOLATILE (default)	Can change every call, even with same input. May read/write DB.	Must be executed exactly as written.	CREATE TABLE IF NOT EXISTS space_travel.observation_log(id bigserial PRIMARY KEY, objectname text NOT NULL, observed_at timestampz NOT NULL); CREATE OR REPLACE FUNCTION space_travel.log_observation(p_object text) RETURNS timestampz AS \$\$ DECLARE ts timestampz := clock_timestamp(); BEGIN INSERT INTO space_travel.observation_log(objectname, observed_at) VALUES (p_object, ts); RETURN ts; -- calling twice (even with same arg) can return different values END; \$\$ LANGUAGE plpgsql VOLATILE;

1. Try to write functions so they can be **IMMUTABLE** → best performance & indexability.
2. If that's not honest (it reads tables/time/parameters), mark **STABLE**.
3. Only if it changes per call or has side effects, mark **VOLATILE**.

Slide 64

64

STRUCTURE: STORED PROCEDURE

- CREATE OR REPLACE PROCEDURE <procedure_name>(<arguments>)
- AS <block-of-code>
- LANGUAGE <implementation-language>;
- **Procedures** are mainly used for transactions and allow COMMIT and ROLLBACK.

Slide 65

65

PRACTICAL SCRIPT FOR TESTING

```
BEGIN;
code and testing
ROLLBACK;
```

DROP [procedure|function|trigger] name
CREATE [procedure|function|trigger] name
ALTER [procedure|function|trigger] name

CREATE OR REPLACE [procedure|function|trigger] name

Slide 66

66

STORED PROCEDURE WITH TRANSACTION

```
CREATE OR REPLACE PROCEDURE space_travel.update_distance(p_objectname text,
p_distance numeric)
AS $$
BEGIN
    -- update distance
    UPDATE space_travel.celestial_object
    SET distance = p_distance
    WHERE objectname = p_objectname;
    -- If the update succeeds, commit
    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        -- Rollback everything if any error occurs
        ROLLBACK;
        RAISE NOTICE 'Transaction rolled back for %', p_objectname;
END;
$$
LANGUAGE plpgsql;

CALL space_travel.update_distance('Earth', 100);
CALL space_travel.update_distance('A', 100);
```

Slide 67

67

TRIGGERS

- A **trigger** is code stored in the database catalog that the DBMS automatically executes in response to certain operations (such as INSERT, UPDATE, or DELETE) when predefined conditions are met.
- Unlike functions or procedures invoked explicitly, triggers are fired implicitly by the system when the specified event occurs.
- PostgreSQL implements the standard functionality—but also includes several **PostgreSQL-specific enhancements**.

Slide 68

68

TRIGGER COMPONENTS IN POSTGRES

- **Trigger Function**
 - A user-defined PL/pgSQL function with **no arguments** that must return type TRIGGER.
 - It contains the logic that executes when triggered, using context variables like **NEW, OLD, TG_OP, TG_LEVEL**, etc
- **Trigger Definition**
 - Created with CREATE TRIGGER, it specifies:
 - **Timing** — BEFORE, AFTER, or INSTEAD OF.
 - **Event(s)** — INSERT, UPDATE, DELETE, or TRUNCATE.
 - **Target** — table, view, or foreign table.
 - **Level** — FOR EACH ROW (row-level) or FOR EACH STATEMENT (statement-level).
 - Optionally, a **WHEN (condition)** clause to filter when the trigger should fire.
 - References the trigger function using EXECUTE FUNCTION (modern) or legacy EXECUTE PROCEDURE.

Slide 69

69

TRIGGER FUNCTION EXAMPLE (AUDIT LOG)

- Create an audit log for the celestial_object table. All INSERT, UPDATE, and DELETE operations should be logged automatically, capturing the values before and after the change.
- Create the audit table

```
CREATE TABLE IF NOT EXISTS space_travel.celestial_object_audit (
    audit_id SERIAL PRIMARY KEY,
    changed_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    changed_by TEXT NOT NULL DEFAULT current_user,
    operation CHAR(1) NOT NULL CHECK (operation IN ('I','U','D')),
    objectname VARCHAR(10) NOT NULL,
    satellite_of VARCHAR(10),
    distance NUMERIC(10,3),
    diameter NUMERIC(7)
);
```

Slide 70

70

TRIGGER FUNCTION EXAMPLE (AUDIT LOG)

Create the trigger function

```
CREATE OR REPLACE FUNCTION space_travel.log_celestial_changes()
RETURNS trigger
AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO space_travel.celestial_object_audit (
            operation, objectname, satellite_of, distance, diameter) VALUES (
                'I', NEW.objectname, NEW.satellite_of, NEW.distance, NEW.diameter);
        RETURN NEW;
    ELSEIF TG_OP = 'UPDATE' THEN
        INSERT INTO space_travel.celestial_object_audit (
            operation, objectname, satellite_of, distance, diameter) VALUES (
                'U', NEW.objectname, NEW.satellite_of, NEW.distance, NEW.diameter);
        RETURN NEW;
    ELSEIF TG_OP = 'DELETE' THEN
        INSERT INTO space_travel.celestial_object_audit (
            operation, objectname, satellite_of, distance, diameter) VALUES (
                'D', OLD.objectname, OLD.satellite_of, OLD.distance, OLD.diameter);
        RETURN OLD;
    END IF;
    RETURN NULL; -- fallback, shouldn't be reached
END;
$$
LANGUAGE plpgsql;
```

- TG_OP is a special **trigger context variable** (of type text) automatically available within PL/pgSQL trigger functions.
- It holds the **operation name** that fired the trigger. It can be one of:
 - 'INSERT'
 - 'UPDATE'
 - 'DELETE'
 - 'TRUNCATE'
- In **row-level trigger functions**, PostgreSQL provides two special variables: **NEW** and **OLD**
 - **NEW**: A RECORD holding the **new version** of the row being inserted or updated.
 - **OLD**: A RECORD containing the **previous version** of the row being updated or deleted.

Slide 71

71

TRIGGER FUNCTION EXAMPLE (AUDIT LOG)

- Attach the trigger to the table
- ```
CREATE TRIGGER trg_celestial_object_audit
AFTER INSERT OR UPDATE OR DELETE ON space_travel.celestial_object
FOR EACH ROW
EXECUTE FUNCTION space_travel.log_celestial_changes();
```
- Test trigger

```
INSERT INTO celestial_object VALUES ('Test', NULL, 100, 100);

UPDATE celestial_object SET distance = 200
WHERE objectname = 'Test';

DELETE FROM celestial_object
WHERE objectname = 'Test';

SELECT * FROM space_travel.celestial_object_audit;
```

| audit_id | changed_at              | changed_by | operation | objectname | satellite_of | distance | diameter |
|----------|-------------------------|------------|-----------|------------|--------------|----------|----------|
| 1        | 2020-09-22 17:28:55.888 | postgres   | I         | Test       |              | 100.000  | 100      |
| 2        | 2020-09-22 17:28:55.888 | postgres   | U         | Test       |              | 200.000  | 100      |
| 3        | 2020-09-22 17:28:55.888 | postgres   | D         | Test       |              | 200.000  | 100      |

Slide 72

72

## SYNTAX

- CREATE [ OR REPLACE ] [ CONSTRAINT ] TRIGGER name ( BEFORE | AFTER | INSTEAD OF ) { event [ OR ... ] }
- ON table\_name
- [ FROM referenced\_table\_name ]
- [ ( NOT DEFERRABLE | [ DEFERRABLE ] [ INITIALLY IMMEDIATE | INITIALLY DEFERRED ] ) ]
- [ REFERENCING ( { OLD | NEW } TABLE [ AS ] transition\_relation\_name ) [ ... ] ]
- [ FOR [ EACH ] { ROW | STATEMENT } ]
- [ WHEN ( condition ) ]
- EXECUTE ( FUNCTION | PROCEDURE ) function\_name ( arguments )
- where event can be one of:
  - INSERT
  - UPDATE [ OR column\_name { , ... } ]
  - DELETE
  - TRUNCATE
- URL: <https://www.postgresql.org/docs/current/sql-createtrigger.html>

73

## SYNTAX

- CREATE/ALTER/DROP TRIGGER
- ALTER TABLE ... ENABLE/DISABLE TRIGGER .
- GRANT/REVOKE TRIGGER ON TABLE ... TO ...
- ( CREATE EVENT TRIGGER – DDL – PostgreSQL specific)

74

## ANSI SQL STANDARD

- ANSI SQL allows more actions than a trigger function (udf : user defined function)
- Example

```
create trigger delete_players
after delete on player for each row
begin
 delete from match
 where player_nr = old.player_nr;
end ;
```

--no reference to udf, but directly the sql code
- PostgreSQL **does not support inline SQL-procedural logic** directly inside a CREATE TRIGGER statement. Instead, every trigger must reference a **trigger function** defined separately

75

## DIFFERENCES BETWEEN PRODUCTS

- There are differences between products :
  - Multiple triggers on 1 table ? And what about the order ?
  - Can a trigger activate another trigger ? (waterfall/domino !)
  - What is a trigger allowed to do ?
  - How are (more complex) triggers processed correctly ?
  - ..

76

## USAGE?

- Think event-driven
- Examples :
  - (Integrity) rules
  - Audit
  - Consistency
  - Security
  - Calculate derived values
  - (Data) validation
  - ..

77

## ADVANTAGES

- Maintainability
  - Eg. Faster execution due to more instructions in macro
- Processing speed
  - Eg. minimises network traffic
- "Precompilation" with stored procedures, functions and triggers
  - Planning and caching
- Works in different host languages

78

## CONS

- More upfront thinking about architecture and code organisation.
- Generally speaking as with other languages : logical errors vs syntax errors
  - Unexpected side effects when using (many) (overlapping) triggers
  - More difficult to debug
  - Black box – hidden complexity

79

## PROCEDURAL SQL VS. APPLICATION LAYER LOGIC

- **Procedural SQL (Stored Procedures / Triggers) – when to use:**
  - Heavy **data processing** inside the database (minimizes network overhead, leverages set operations).
  - Need centralized logic for **audit, validation, or security** (e.g., access control, input checks).
  - Want tight **transaction control** embedded (e.g., commit or rollback inside procedures).
  - Multiple different applications share the same logic — **reusability and consistency** across apps.
- **Application Layer Logic – when to use:**
  - Complex, evolving **business workflows**, especially across multiple services/APIs. Better supported by modern languages, frameworks.
  - Teams need **scalable, maintainable code with version control integration**, CI/CD, and testing pipelines.
  - You want **portability** across different databases—avoid vendor lock-in due to stored procedure dialects.

80

## PROCEDURAL SQL VS. APPLICATION LAYER LOGIC

Most real-world systems adopt a **hybrid approach**:

- Use **procedural SQL** for high-performance, data-intensive routines (filters, audits, validations).
- Keep **application logic** for business workflows, orchestration, UI logic, external integrations, and maintainability.

81



## OPO: Data Management

Aimée Lynn Backiel  
Wim Bertels  
Alain Van Der Jeught

Module B: Advanced SQL

JOINS

TI / BCS

Attribution-NonCommercial-ShareAlike 4.0 Unported Licence

2025-2026

82

## SYMBOLS USED IN THIS COURSE

| Symbol                                                                              | Category      | Typical Slide Use                                       |
|-------------------------------------------------------------------------------------|---------------|---------------------------------------------------------|
|  | Positioning   | Positioning, purpose                                    |
|  | Objective     | Objective, goal                                         |
|  | Attention     | Warnings, key pitfalls, critical notes                  |
|  | Theory        | Concept explanations, definitions                       |
|  | Practice      | Exercises, hands-on labs                                |
|  | Clip          | Demo, Clip                                              |
|  | Poll          | Live questions / Quiz                                   |
|  | Informational | Neutral notes, references, reading tips. Not examinable |

83

## PURPOSE

- **Understand** advanced ways to control how tables are joined in SQL.
- **Learn** how additional conditions (in ON and WHERE) affect join results.
- **Explore** specialized join syntax (JOIN USING, NATURAL JOIN).
- **Distinguish** between **Equi-Join** and **Theta Join**.
- Apply **LATERAL JOIN** for correlated subqueries in the FROM clause.

84

## OBJECTIVES

- **Write** joins with additional filtering conditions in the ON clause vs. the WHERE clause, and explain the difference in results.
- **Use** JOIN USING to simplify column matching when names are identical.
- **Apply** NATURAL JOIN and understand when it's appropriate (and risky).
- **Differentiate** between Equi-Joins (equality conditions) and Theta Joins (other comparison operators).
- **Construct** queries with LATERAL JOIN to reference columns from earlier tables in subqueries.

Slide 85

85

## JOINS

- This presentation builds further on the "SQL Foundation Recap" presentation and assumes that material has been mastered.
- In this presentation, the following topics will be addressed:
  - Additional conditions
  - JOIN USING
  - NATURAL JOIN
  - Equi/Theta JOIN
  - LATERAL JOIN

Slide 86

86

## SEARCH\_PATH

- SHOW search\_path;
- SET search\_path = public, myschema;
- Tables with the **same name** in different schemas can lead to confusion. Always consider **explicitly specifying the schema name** for clarity.

Slide 87

87

## COMBINING TABLES - IMPLICIT

- Implicit join = matches rows between 2 or more tables separated by comma
- Match condition = in WHERE clause
- Question: Which journeys has each client participated in, along with their client number and last name?  

```
SELECT c.client_nr, c.lastname, p.journey_nr
FROM client c, participation p
WHERE c.client_nr = p.client_nr
```

The WHERE clause is important as without a cartesian product will be generated. A cartesian product will show each row in the left table once for every row in the right table (CROSS JOIN). This may have an important impact on performance.

Slide 88

88

## COMBINING TABLES - EXPLICIT

- The clause **JOIN** explicitly matches rows between 2 or more tables
- Different types: INNER, LEFT, RIGHT, FULL OUTER, CROSS JOIN
- Match condition: in ON clause
- Question: Which journeys has each client participated in, along with their client number and last name?  

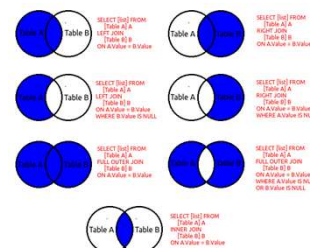
```
SELECT c.client_nr, c.lastname, p.journey_nr
FROM client c INNER JOIN participation p ON c.client_nr = p.client_nr;
```

The ON operator is important as without a cartesian product will be generated. A cartesian product will show each row in the left table once for every row in the right table. This may have an important impact on performance.

Slide 89

89

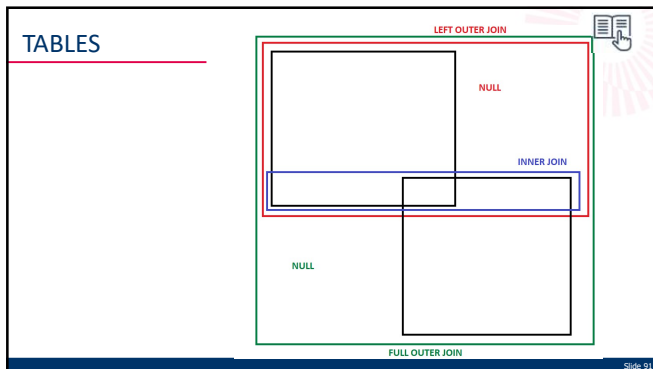
## JOIN TYPES



90

Slide 90

90



91

## JOIN CONDITIONS: FROM VERSUS WHERE

- a JOIN statement can have **additional conditions** beyond just matching the keys between tables. These extra conditions can be placed either:
  - o In the **ON clause** (affects which rows get joined)
  - o In the **WHERE clause** (affects which joined results are kept afterward)

```

SELECT team.player_nr, team.team_nr, fine.payment_nr
FROM teams
LEFT OUTER JOIN fine ON (team.player_nr = fine.player_nr)
AND division = 'first'; -- additional join condition
versus
SELECT team.player_nr, team.team_nr, fine.payment_nr
FROM team
LEFT OUTER JOIN fine ON (team.player_nr = fine.player_nr)
WHERE division = 'first'; -- filter after join

```

What is the difference in the resultsets of both queries?

Slide 92

92

## EXAMPLE

- Question: List all players, including their player number, who have never been fined 50 euros.
- Provide 2 equivalent solutions
  - o One without an implicit/explicit join
  - o One with an explicit JOIN
- "is equivalent to" — means both queries return the *same result set* for all possible valid databases.

Slide 93

93

## SOLUTION 1

Question: List all players, including their player number, who have never been fined 50 euros.

```

SELECT player_nr
FROM player
WHERE player_nr NOT IN
 (SELECT player_nr
 FROM fine
 WHERE amount = 50);

```

- Uncorrelated solution

Slide 94

94

## SOLUTION 2

Question: List all players, including their player number, who have never been fined 50 euros.

```

SELECT player.player_nr
FROM player
LEFT OUTER JOIN fine
 ON player.player_nr = fine.player_nr AND fine.amount = 50
WHERE fine.player_nr IS NULL;

```

- Logic:
  - o LEFT OUTER JOIN keeps all players.
  - o fine.amount = 50 is applied **while joining**.
  - o If a player has **no 50 euro fine**, then fine.player\_nr will be **NULL**.
  - o The WHERE fine.player\_nr IS NULL filter removes only those with a 50 euro fine.

Slide 95

95

## SOLUTION 3?

Question: List all players, including their player number, who have never been fined 50 euros.

```

SELECT player.player_nr
FROM player
LEFT OUTER JOIN fine ON player.player_nr = fine.player_nr
WHERE fine.amount <> 50 OR fine.amount IS NULL;

```

- Why is this solution not a correct answer to the question?

Slide 96

96

## JOIN USING

- **JOIN USING** is a shorthand for joining tables when the column name is the same in both tables.
- It eliminates the need to specify table1.col = table2.col in the ON clause.
- Example:

```
SELECT player.player_nr
FROM player
INNER JOIN fine
ON player.player_nr = fine.player_nr
```

Is equivalent to

```
SELECT player.player_nr
FROM player
INNER JOIN fine
USING (player_nr)
```

trying USING + extra join condition does not work  
USING cannot be used with NATURAL joins and CROSS joins

Slide 97

97

## NATURAL JOIN

- A NATURAL JOIN automatically matches and joins tables **on all columns with the same name** in both tables, returning only one copy of each matching column.
- Example  

```
SELECT *
FROM team NATURAL INNER JOIN fine
WHERE division = first;
```

Slide 98

98

## NATURAL JOIN

- Joins automatically on all columns with the same name.
- No need to specify ON or USING.
- Avoids duplicate join columns in the output.
- Can be risky: join criteria are determined automatically, which may cause unexpected results if unintended column names match.
- Tip: Use only when you are certain that the common columns are the correct join keys.
- Also works with other JOINS (e.g. NATURAL RIGHT OUTER JOIN)

Slide 99

99

## NATURAL JOIN VS JOIN USING

| Feature                  | NATURAL JOIN           | JOIN USING                 |
|--------------------------|------------------------|----------------------------|
| Column selection         | All same-named columns | Only columns you specify   |
| Control over columns     | None                   | Full control               |
| Risk of unexpected joins | Higher                 | Lower                      |
| Syntax simplicity        | Shortest               | Still concise but explicit |

- Use JOIN USING when you want explicit control over join keys.
- Use NATURAL JOIN only when you are certain that all same-named columns should be part of the join.

Slide 100

100

## EQUI/THETA JOIN

- **Equi Join:** a type of **Theta Join** that uses the equality operator (=) in the join condition.
- **Theta Join:** any join that uses a comparison operator (=, <, >, <=, >=, <>, etc.) in the join condition.

Slide 101

101

## EQUI/THETA JOIN EXAMPLE 1

Question: Count how many players there are with a different number. Show the player number, name and count.

Slide 102

102

### EQUI/THETA JOIN EXAMPLE 1 SOLUTION

Question: Count how many players there are with a different number. Show the player number, name and count.

```
SELECT p1.player_nr, p1.last_name, count(p2.player_nr)
FROM player p1
INNER JOIN player p2 ON (p1.player_nr <> p2.player_nr)
GROUP BY p1.player_nr, p1.last_name;
```

Slide 103

103

### EQUI/THETA JOIN EXAMPLE 2

Question: Count how many players there are with an equally long name. Show the player number, name and count.

Slide 104

104

### EQUI/THETA JOIN EXAMPLE 2 SOLUTION

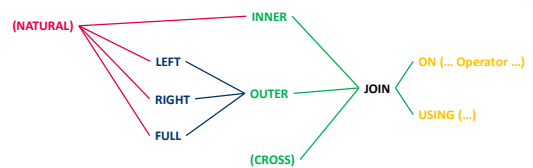
Question: Count how many players there are with an equally long name. Show the player number, name and count.

```
SELECT p1.player_nr, p1.last_name, count(p2.player_nr)
FROM player p1
INNER JOIN player p2 ON (p1.player_nr <> p2.player_nr)
AND LENGTH(p1.last_name) = LENGTH(p2.last_name)
GROUP BY p1.player_nr, p1.last_name;
```

Slide 105

105

### SUMMARY



() : USING and ON cannot be used

Slide 106

106

### LATERAL JOIN

- Question: For each player, list their player number and the amount(s) of their fines.

```
SELECT p.player_nr, playerfine.amount
FROM player p,
 (SELECT f.amount
 FROM fine f
 WHERE f.player_nr = p.player_nr) As playerfine;
```

Does this query generate a resultset?

Slide 107

107

### LATERAL JOIN

- What is the problem?
- The parser complains because p.playernr hasn't been introduced yet for the subquery.

 SQL Error [42P01]: ERROR: invalid reference to FROM-clause entry for table "p"  
Detail: There is an entry for table "p", but it cannot be referenced from this part of the query.

- In standard SQL, a subquery in the FROM clause is treated as **independent** of the other tables listed there.

Slide 108

108

## LATERAL JOIN

- Subqueries that appear in **FROM** can be preceded by the keyword **LATERAL**.
- This allows subqueries to refer to columns from preceding FROM items.
- In other words, LATERAL allows subqueries in the FROM clause to behave like correlated subqueries, giving them access to columns from earlier tables in the join order.
- When you have **multiple subqueries** in the FROM clause and each one needs to reference columns from the outer query (e.g., a table like *players*), you must write **LATERAL explicitly for each subquery**.

Slide 109

109

## LATERAL JOIN

- Via **LATERAL**, a table expression in the FROM clause can refer to columns from **previous items** in the same FROM clause, including:
  - A **previous table reference**
  - A **previous subquery**
  - A **previous set-returning function (SRF)**
    - When using a **set-returning function** in FROM that *isn't* preceded by JOIN, most SQL dialects (like PostgreSQL) treat it as **LATERAL by default**, so you can omit the LATERAL keyword.
    - However, for **subqueries** or when clarity matters, explicitly writing LATERAL is often safer and more readable.

Slide 110

110

## LATERAL JOIN SOLUTION

- Question: For each player, list their player number and the amount(s) of their fines.

```
SELECT p.player_nr, playerfine.amount
FROM player p,
 LATERAL (SELECT f.amount
 FROM fine f
 WHERE f.player_nr = p.player_nr) As playerfine;
```

Slide 111

111

## LATERAL JOIN PITFALLS

- **Performance Issues**
  - **Correlated subqueries** inside LATERAL are evaluated **per row** of the outer table.
  - If the subquery is expensive or unindexed, it can **drastically slow down** the query.
- **Non-intuitive Join Semantics**
  - LATERAL subqueries **depend on join order**.
  - Unlike typical joins, the subquery can't be reordered freely by the optimizer, which may affect performance or results.

Slide 112

112

## LATERAL JOIN EXAMPLE

- Question: show age, including client number, name and birthdate, for each client (with a lateral join).

Slide 113

113

## LATERAL JOIN EXAMPLE SOLUTION 1

- Question: show age, including client number, name and birthdate, for each client (with a lateral join).

```
SELECT c.client_nr, c.lastname, c.birthdate, lc.age
FROM client c
LEFT OUTER JOIN LATERAL (SELECT age(c.birthdate) as age) AS lc
ON (lc.age=age(c.birthdate));
```

Slide 114

114

## LATERAL JOIN EXAMPLE

- Is the join condition "**ON** (lc.age=age(c.birthdate))" necessary?

Slide 115

115

## LATERAL JOIN EXAMPLE

- The query is equivalent to:

```
SELECT c.client_nr, c.lastname, c.birthdate, lc.age
FROM client c
LEFT JOIN LATERAL
(SELECT age(c.birthdate) as age) AS lc
ON TRUE;
```

- The ON true means *every row* from client matches *every row* from the subquery output for that same client row.
- Because the lateral subquery only produces **one row per client**, the Cartesian product in this case is just  $1 \times 1 \Rightarrow$  one joined row per client.

Slide 116

116

## LATERAL JOIN CONDITION

When a FROM item contains LATERAL cross-references, evaluation proceeds as follows:  
for each row of the FROM item providing the cross-referenced column(s), or set of rows of multiple FROM items providing the columns, the LATERAL item is evaluated using that row or row set's values of the columns. The resulting row(s) are joined as usual with the rows they were computed from. This is repeated for each row or set of rows from the column source table(s).

- > Cartesian product behaves here as a foreach loop

Slide 117

117

## LATERAL JOIN TYPES

- Choosing the JOIN type with a correlated subquery (LATERAL)
  - Allowed: INNER JOIN LATERAL, LEFT JOIN LATERAL, or CROSS JOIN LATERAL.
  - Not allowed: RIGHT JOIN or FULL JOIN
- Key difference between LEFT and INNER:
  - LEFT JOIN**: preserves all rows from the left table, even when the correlated subquery yields nothing.
  - INNER JOIN**: discards left rows if the subquery yields nothing or the ON condition fails.

Slide 118

118

## LATERAL JOIN EXERCISE

- Question: Provide for each journey, the two smallest objects visited. Show all data.
  - Break down the question by checking for 1 object
- ```
SELECT *
FROM visit NATURAL INNER JOIN celestial_object
WHERE journey_nr = 32
ORDER BY diameter;
```

objectname	journey_nr	sequence_number	duration_of_visit	satellite_nr	distance	diameter
Chernobyl	32	3	0	Mars	23,400	7
Phobos	32	2	0	Mars	9,270	14
Moon	32	5	2	Earth	384,400	3,476
Moon	32	1	0	Earth	384,400	3,476
Mars	32	4	3	Sun	227,900,000	6,794

Slide 119

119

LATERAL JOIN EXERCISE

- Question: Provide for each journey, the two smallest objects visited. Show all data.

```
SELECT *
FROM journey j LEFT JOIN LATERAL
(SELECT *
FROM visit v NATURAL INNER JOIN celestial_object co
WHERE v.journey_nr = j.journey_nr
ORDER BY co.diameter
FETCH FIRST 2 ROWS ONLY) AS lj
ON TRUE;
```

Slide 120

120



OPO: Data Management

Aimée Lynn Backiel
Wim Bertels
Alain Van Der Jeught

Module B: Advanced SQL

SET Operators

TI / BCS Attribution-NonCommercial-ShareAlike 4.0 Unported Licence 2025-2026

121

SYMBOLS USED IN THIS COURSE

Symbol	Category	Typical Slide Use
	Positioning	Positioning, purpose
	Objective	Objective, goal
	Attention	Warnings, key pitfalls, critical notes
	Theory	Concept explanations, definitions
	Practice	Exercises, hands-on labs
	Clip	Demo, Clip
	Poll	Live questions / Quiz
	Informational	Neutral notes, references, reading tips. Not examinable

122

PURPOSE

- Combine results from multiple queries into a single result set.
- Support relational algebra operations directly in SQL.
- Useful for comparing datasets (e.g., common rows, differences, or consolidations).
- Allow handling duplicates flexibly (with/without ALL).
- Enhance query readability versus complex JOIN alternatives.

123

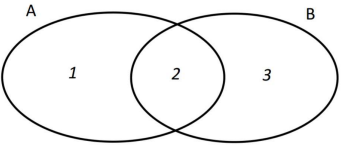
OBJECTIVES

- Understand** the purpose of set operators in SQL.
- Differentiate** between:
 - UNION vs UNION ALL
 - INTERSECT vs INTERSECT ALL
 - EXCEPT vs EXCEPT ALL
- Apply** set operators to solve practical query problems.
- Predict** result sets, especially with duplicates.
- Recognize** when to use set operators versus JOINS.

124

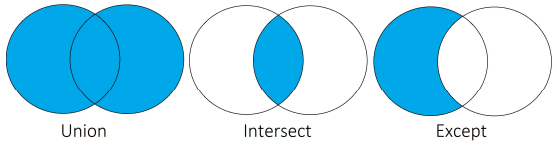
SET OPERATORS

- Combining results of individual SELECT instructions
- Capabilities:
 - UNION
 - INTERSECT
 - EXCEPT
 - UNION ALL
 - INTERSECT ALL
 - EXCEPT ALL



125

MATHEMATICS - CALCULUS

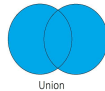


126

UNION

- The **UNION operator** combines the result sets of two or more SELECT queries into a single result set.
- It **returns only distinct rows** (removes duplicates) by default.
- Example: List the player number of each player who has paid at least one fine, or who is captain of a team, or both.

```
SELECT player_nr
FROM fine
UNION
SELECT player_nr
FROM team;
```



Slide 127

127

UNION RULES

- Each SELECT must return the same number of columns.
- Corresponding columns must have compatible data types.
- By default, UNION removes duplicates (UNION ALL keeps them), hence DISTINCT should not be used.
- Only one ORDER BY may appear, and it applies to the final combined result set.

Slide 128

128

UNION AND GROUP BY

- Example: Combining UNION and GROUP BY to perform subtotal and total calculations.
- Question: How many games (wins + losses) were recorded per player within each team, with a subtotal per team and a grand total across all teams?

Slide 129

129

UNION AND GROUP BY

- Question: How many games (wins + losses) were recorded per player within each team, with a subtotal per team and a grand total across all teams?

```
SELECT CAST(team_nr AS char(4)) AS teamnr, CAST(player_nr AS char(4)) AS playernr, SUM(nr_sets_won + nr_sets_lost) AS total
FROM match
GROUP BY team_nr, player_nr
UNION
SELECT CAST(team_nr AS char(4)), 'subtotal', SUM(nr_sets_won + nr_sets_lost)
FROM match
GROUP BY team_nr
UNION
SELECT 'total', 'total', SUM(nr_sets_won + nr_sets_lost)
FROM match
ORDER BY 1,2;
```

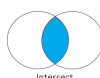
Slide 130

130

INTERSECT

- INTERSECT combines the results of two SELECT statements and outputs the **intersection**:
 - Only rows that appear in **both result sets** are included.
 - By default, duplicates are removed (like UNION).
- Example: List the player numbers of players who are captain of a team and have paid at least one fine.

```
SELECT player_nr
FROM team
INTERSECT
SELECT player_nr
FROM fine;
```



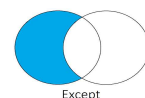
Slide 131

131

EXCEPT

- **EXCEPT** returns rows from the **first query** that do **not appear** in the **second query**.
- Removes duplicates by default (like UNION and INTERSECT).
- Example: List the player numbers of players who have paid at least one fine and are not captain of a team.

```
SELECT player_nr
FROM fine
EXCEPT
SELECT player_nr
FROM team;
```



Slide 132

132

ALL = RETAIN DUPLICATES

- Default: duplicate rows are removed
- ALL variant: to keep duplicate rows
 - UNION ALL
 - INTERSECT ALL
 - EXCEPT ALL
- Example: List the player numbers of players who have paid at least one fine and are not captain of a team. Maintain double rows.

```
SELECT player_nr
FROM fine
EXCEPT ALL
SELECT player_nr
FROM team;
```

Slide 133

133

NULL

- Normally, In comparisons (=), **NULL is never equal to NULL**.
- But in set operators SQL treats NULL differently.
- When checking for duplicates or overlaps, **two NULLs in the same column position are considered equal**.

```
SELECT CASE
  WHEN NULL = NULL THEN 'equal'
  ELSE 'not equal'
END;
```

```
SELECT NULL AS column1
UNION
SELECT NULL AS column1;
```

Slide 134

134

COMBINATIONS

- Multiple set operators (UNION, INTERSECT, EXCEPT) can be combined in a single query.
- By default, SQL evaluates them **from left to right** (like arithmetic without parentheses).
- **INTERSECT** has the **highest precedence**.
- **UNION** and **EXCEPT** have **equal precedence**, evaluated **left to right**.
- **Brackets (parentheses)** can be used to **change the evaluation order** and control how the intermediate result sets are combined.

Slide 135

135

EXAMPLE

- Example: List the player numbers of players who have paid at least one fine but are not captain of a team, as well as the players from Hove.

```
(SELECT player_nr
FROM fine
EXCEPT
SELECT player_nr
FROM team)
UNION
SELECT player_nr
FROM player
WHERE city = 'Hove';
```

Slide 136

136

JOINS VS SET OPERATORS

- **Joins**
 - **Purpose:** Combine **columns** from multiple tables based on related keys.
 - **Use when:** You need information from **different tables** in a single row.
 - **Result:** A *wider* table (more columns).
 - **Operators:** INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN.
- **Set Operators**
 - **Purpose:** Combine **rows** from multiple queries into a single result set.
 - **Use when:** You need a **union, intersection, or difference** between lists of rows.
 - **Result:** A *longer* table (more rows).
 - **Operators:** UNION, UNION ALL, INTERSECT, EXCEPT.

Slide 137

137

OPO: Data Management

Aimée Lynn Backiel
Wim Bertels
Alain Van Der Jeught

Module B: Advanced SQL Compare Patterns

TI / BCS

Attribution-NonCommercial-ShareAlike 4.0 Unported Licence

2025-2026

138

SYMBOLS USED IN THIS COURSE

Symbol	Category	Typical Slide Use
	Positioning	Positioning, purpose
	Objective	Objective, goal
	Attention	Warnings, key pitfalls, critical notes
	Theory	Concept explanations, definitions
	Practice	Exercises, hands-on labs
	Clip	Demo, Clip
	Poll	Live questions / Quiz
	Informational	Neutral notes, references, reading tips. Not examinable

Slide 139

139

PURPOSE

- Understand how to **search and filter text values** using patterns.
- Distinguish between **wildcard-based** (LIKE, ILIKE), **standardized** (SIMILAR TO), and **Postgres extensions** (POSIX regex).
- Learn **case-sensitive vs case-insensitive** comparisons.
- Handle **special characters** (% , _) with escaping.
- Apply the right tool for **simple searches vs complex text-matching needs**.

Slide 140

140

OBJECTIVES

- **Use** LIKE and ILIKE for basic pattern matching.
- **Apply** ESCAPE to match literal % and _.
- **Explain** the difference between LIKE, ILIKE, SIMILAR TO

Slide 141

141

COMPARE PATTERNS

- LIKE - operator (SQL default)
- SIMILAR TO - operator (SQL standard since 1999)
- Regular expressions, POSIX form, not a SQL standard,
- <http://www.postgresql.org/docs/current/static/functions-matching.html>

Slide 142

142

LIKE OPERATOR

- LIKE is used for **pattern matching** in SQL.
 - Special characters:
 - % → matches any sequence of characters (0 or more).
 - _ → matches exactly one character.
- Supplement
 - WHERE name LIKE '%/_%' ESCAPE '/'
 - Escaping characters ensures that they are used as they are, without interpreting them. This is useful for indicating _ and % as explicit characters in a query
 - You can also use NOT LIKE to exclude patterns.
 - E.g WHERE name NOT LIKE 'br%'

Slide 143

143

SIMILAR TO OPERATOR

- Like LIKE operator with % and _ and ESCAPE
- Extras as in 'regular' expressions:
 - | stands for or
 - * represents a possible repetition (0,1,...)
 - + represents a possible repetition (1,2,...)
 - ? stands for zero or 1 repetition (0,1)
 - {m} exactly m repetitions
 - {m,} m or more repetitions
 - {m,n} at least m, maximum n repetitions
 - () to take together
 - [] analogous to ordinary regular expressions (posix) e.g. [abc]

Slide 144

144

SIMILAR TO OPERATOR: EXAMPLES

- 'abc' SIMILAR TO 'abc'
- 'abc' SIMILAR TO 'a'
- 'abc' SIMILAR TO '%(b|d)%'
- 'abc' SIMILAR TO '(b|c)%'
- 'abbc' SIMILAR TO 'ab+ _'
- 'a' SIMILAR TO 'ab+'
- 'a' SIMILAR TO 'ab*'
- 'a' SIMILAR TO 'ab?'
- 'abbz' SIMILAR TO 'ab*[xyz]'
- 'aba' SIMILAR TO '(ab*){2,}'
- 'abaaaz' SIMILAR TO '(ab*){1,2}[az]+'

Slide 145

145

SIMILAR TO OPERATOR: EXAMPLES

- 'abc' SIMILAR TO 'abc' true
- 'abc' SIMILAR TO 'a' false
- 'abc' SIMILAR TO '%(b|d)%' true
- 'abc' SIMILAR TO '(b|c)%' false
- 'abbc' SIMILAR TO 'ab+ _' true
- 'a' SIMILAR TO 'ab+' false
- 'a' SIMILAR TO 'ab*' true
- 'abbz' SIMILAR TO 'ab*[xyz]' true
- 'aba' SIMILAR TO '(ab*){2,}' true
- 'abaaaz' SIMILAR TO '(ab*){1,2}[az]+' true

Slide 146

146

UPPERCASE AND LOWERCASE

- PostgreSQL specific:
 - ILIKE: case-insensitive (LIKE: case-sensitive)
 - WHERE name ILIKE 'Tom'; -- matches 'Tom', 'TOM', 'ToM', etc.
- General ANSI SQL:
 - e.g. use a function to convert everything to lowercase

```
SELECT *
FROM table t
WHERE lower(t.field) SIMILAR TO 'somelowercasestring';
```
- <http://www.postgresql.org/docs/current/interactive/functions-string.html>

Slide 147

147

OTHER OPERATORS

- BETWEEN
 - WHERE x between 1 and 100
- OVERLAPS
 - WHERE (date1,date2) OVERLAPS (dateA, dateB)
- IS NULL
 - WHERE x IS NULL -- do not use x = NULL
- NOT (negation)

Slide 148

148

SOME CONDITIONAL FUNCTIONS

- COALESCE
 - Returns the **first non-NULL value** in the list.
 - If all arguments are NULL, it returns NULL.
 - Example: SELECT COALESCE(amount, 0); -- ensures that NULL amounts are treated as 0
- NULLIF
 - Use NULLIF to turn specific unwanted values into NULLs.
 - SELECT nullif(7,5);
 - SELECT nullif(5,5);
 - SELECT amount / NULLIF(num_fines, 0) -- avoid division by zero
- GREATEST / LEAST
 - Returns the **largest (maximum) / smallest (minimum)** value among its arguments.
 - If any argument is NULL, the result is NULL
 - Different from MAX() / MIN(), which are **aggregates over rows**. GREATEST / LEAST work on values within a single row.
 - SELECT greatest(nr_sets_won, nr_sets_lost), nr_sets_won FROM match;

Slide 149

149

OPO: Data Management

Aimée Lynn Backiel
Wim Bertels
Alain Van Der Jeught

Module B: Advanced SQL Correlated subqueries

TI / BCS

Attribution-NonCommercial-ShareAlike 4.0 Unported Licence

2025-2026

150

SYMBOLS USED IN THIS COURSE

Symbol	Category	Typical Slide Use
	Positioning	Positioning, purpose
	Objective	Objective, goal
	Attention	Warnings, key pitfalls, critical notes
	Theory	Concept explanations, definitions
	Practice	Exercises, hands-on labs
	Clip	Demo, Clip
	Poll	Live questions / Quiz
	Informational	Neutral notes, references, reading tips. Not examinable

Slide 151

151

PURPOSE

- Introduce the concept of **correlated subqueries** and their role in SQL querying.
- Show how correlated queries **depend on values from the outer query** for execution.
- Illustrate typical use cases such as row-by-row comparisons and filtering based on related data.

Slide 152

152

OBJECTIVES

- **Define** a correlated subquery and describe how it operates in relation to the outer query.
- **Identify** scenarios where correlated subqueries are necessary or more intuitive.
- **Write** SQL statements using correlated subqueries in WHERE, SELECT, or FROM clauses.
- **Write** SQL statements using (NOT) EXISTS, ANY, ALL, UNIQUE and OVERLAPS operators.
- **Interpret** query execution flow to understand how the database processes these subqueries.

Slide 153

153

WHAT IS A SUBQUERY?

- A **query inside another query**.
- The inner query runs first, and its **result is passed** to the outer query.
- Can be placed in the WHERE, FROM, or SELECT clauses.
- Subqueries can be **nested** inside other subqueries to multiple levels.
- **Outer query** = the main question you're asking.
- **Inner query (subquery)** = a smaller question whose answer helps answer the main question.

Slide 154

154

WHY DO WE USE SUBQUERIES?

- Break a complex query into **smaller, simpler problems** whose results can be used in the main query, e.g.
- Similar to programming: break down a complex method into **smaller, atomic tasks**.
- Makes SQL easier to **read, write, and maintain**.
- **Filter** based on **calculated results**.
- **Reuse logic** in multiple clauses.
- **Avoid the need** for **storing intermediate results** (temporary tables).

Slide 155

155

NON-CORRELATED VS CORRELATED SUBQUERIES

- **1. Non-Correlated Subquery (see non-correlated subqueries)**
 - **Independent** of the outer query — it can be executed **on its own**.
 - The subquery runs **once**, and its result is then used by the outer query.
 - **Does not** refer to any column from the outer query.
- **2. Correlated Subquery (scope of this presentation)**
 - **Depends on** values from the outer query - it **cannot** be executed **on its own**.
 - The subquery is re-run **for each row** processed by the outer query.
 - References columns from the outer query in its condition.

Slide 156

156

EXAMPLE 1

- Question: For each journey, provide the journey number and celestial object with the longest duration of stay
- What should the inner query calculate?
- What should the outer query return?
- How to handle ties?

Slide 157

157

EXAMPLE 1 SOLUTION

- Question: For each journey, provide the journey number and celestial object with the longest duration of stay

```
SELECT v.journey_nr, v.objectname, v.duration_of_stay
FROM visit AS v
WHERE v.duration_of_stay =
  (SELECT MAX(duration_of_stay)
   FROM visit AS allvisits
   WHERE allvisits.journey_nr = v.journey_nr); → match visits for the
current visit from the outer query.
```

- Remark: If there are ties, you'll get multiple objects for that journey.

Slide 158

158

EXAMPLE 2

- Question: List the player numbers for players who have served as board members more often than they have played matches.

Slide 159

159

EXAMPLE 2 SOLUTION

- Question: List the player numbers for players who have served as board members more often than they have played matches.

```
SELECT b.player_nr
FROM board_member b
GROUP BY b.playernr
HAVING COUNT (*) >
  (SELECT COUNT(*)
   FROM match AS m
   WHERE m.player_nr = b.player_nr);
```

Slide 160

160

EXISTS OPERATOR (IS THERE ANYTHING, OR NOT?)

- **EXISTS** returns **TRUE** if the subquery returns **at least one row**; otherwise **FALSE**.
- The **SELECT list inside the subquery is ignored** (people often use SELECT 1 for clarity).
- **NULLs don't matter**—only the **presence of a row** matters.
- Often used as a **correlated subquery**, where the inner query references the outer query's row.
- NOT EXISTS is the negation (TRUE when the subquery returns **no rows**).

Slide 161

161

EXAMPLE EXISTS OPERATOR

- Question: List all journeys that include a visit to Jupiter, showing the journey number and departure date.
- Alternatively, identify all journeys where a visit to Jupiter **exists**, and display their journey number and departure date.

Slide 162

162

EXAMPLE EXISTS OPERATOR

- Question: List all journeys that include a visit to Jupiter, showing the journey number and departure date.
- Alternatively, identify all journeys where a visit to Jupiter **exists**, and display their journey number and departure date.

```
SELECT j.journey_nr, j.departure_date
FROM journey j
WHERE EXISTS (
  SELECT 1 -- this can be anything
  FROM visit v
  WHERE v.journey_nr = j.journey_nr
  AND v.objectname = 'Jupiter');
```

Slide 163

163

NOT EXISTS OPERATOR (IF THERE IS NOTHING ... THEN ...)

- Question: List all celestial objects that were never visited:

```
SELECT c.objectname
FROM celestial_object c
WHERE NOT EXISTS -- TRUE if the inner query finds no matches
  (SELECT 1 -- can be anything
  FROM visit v
  WHERE v.objectname = c.objectname); -- match visits for
the current celestial object from the outer query.
```

Slide 164

164

AND THIS ONE?

- Question: What is the resultset of this query?

```
SELECT c.objectname
FROM celestial_object c
WHERE NOT EXISTS
  (SELECT v.journey_nr
  FROM visit v);
```

Slide 165

165

ANY AND ALL OPERATORS

- These operators expect a row expression to compare against one or more values (single column!).
 - ALL means the condition must be true for all row returned by the subquery (like 'for all' in mathematics).
 - ANY means the condition must be true for at least one row in the subquery (like 'there exists' in mathematics).

Operator	With ALL (subquery)	With ANY (subquery)
>	Greater than every value in the set	Greater than at least one value in the set
>=	Greater than or equal to every value in the set	Greater than or equal to at least one value
<	Less than every value in the set	Less than at least one value
<=	Less than or equal to every value in the set	Less than or equal to at least one value

Slide 166

166

ANY AND ALL OPERATOR EXAMPLE 1

- Question: Provide the journey number of the longest journey.
- Alternatively: Provide the journey number of the journey whose duration is greater than or equal to all other journeys.

Slide 167

167

ANY AND ALL OPERATOR EXAMPLE 1 SOLUTION

- Question: Provide the journey number of the longest journey.

```
SELECT v1.journey_nr
FROM visit v1
WHERE duration_of_stay >= ALL
  (SELECT v2.duration_of_stay
  FROM visit v2);
```

Slide 168

168

ANY AND ALL OPERATOR, OTHER SOLUTION?

- Is this solution with a correlated subquery a valid solution for the question?

```
SELECT v1.journey_nr
FROM visit v1
WHERE duration_of_stay >= ALL
      (SELECT v2.duration_of_stay
       FROM visit v2
       WHERE v2.journey_nr <> v1.journey_nr);
```

- What is the result if multiple journey have the same duration_of_stay?

Slide 169

169

ANY AND ALL OPERATOR EXAMPLE 2

- Question: List the journey number of all journeys that are shorter than the longest journey.

Slide 170

170

ANY AND ALL OPERATOR EXAMPLE 2

- Question: List the journey number of all journeys that are shorter than the longest journey.

```
SELECT v1.journey_nr
FROM visit v1
WHERE v1.duration_of_stay < ANY
      (SELECT v2.duration_of_stay
       FROM visit v2);
```

Slide 171

171

ANY AND ALL OPERATOR: ATTENTION!

- Question: List planets that are bigger than every one of their satellites

```
SELECT c.objectname
FROM celestial_object c
WHERE satellite_of = 'Sun' AND diameter > ALL
      (SELECT diameter
       FROM celestial_object AS moon
       WHERE moon.satellite_of = c.objectname);
```

A-Z	objectname
	Earth
	Pluto
	Jupiter
	Uranus
	Venus
	Mars
	Saturn
	Neptune
	Mercury

Slide 172

172

ANY AND ALL OPERATOR: ATTENTION!

- Question: List planets that are bigger than every one of their satellites

```
SELECT c.objectname
FROM celestial_object c
WHERE satellite_of = 'Sun' AND diameter < ALL
      (SELECT diameter
       FROM celestial_object AS moon
       WHERE moon.satellite_of = c.objectname);
```

A-Z	objectname
	Venus
	Mercury

Slide 173

173

ANY AND ALL OPERATOR: ATTENTION!

- Question: What are the diameters of the satellites of Mercury and Venus?

```
SELECT diameter
FROM celestial_object
WHERE satellite_of IN ('Mercury', 'Venus');
```

123	diameter
-----	----------

Slide 174

174

(ANY AND ALL OPERATOR) VS NULL VS EMPTY SET

- In SQL, any comparison with NULL returns UNKNOWN (shown as NULL in results), unless another non-NULL value in the set is enough to determine the result of ANY or ALL.
- Special case: If the subquery returns no rows then:
 - Returns ALL: TRUE;
 - Returns ANY: FALSE;
- If you want to avoid UNKNOWN from NULLs, filter them out in the subquery.

Slide 175

175

ANY AND ALL OPERATOR: ATTENTION!

- Question: List planets that are bigger than every one of their satellites

```
SELECT c.objectname
FROM celestial_object c
WHERE satellite_of = 'Sun'
AND EXISTS
  (SELECT 1
   FROM celestial_object AS moon
   WHERE moon.satellite_of = c.objectname)
AND diameter > ALL
  (SELECT diameter
   FROM celestial_object AS moon
   WHERE moon.satellite_of = c.objectname);
```



objectname
Mars
Earth
Jupiter
Pluto
Uranus
Neptune
Saturn

Slide 176

176

UNIQUE OPERATOR

- Checks whether a subquery produces **duplicate rows**.
- Question: List the players who have paid exactly one penalty.

```
SELECT player_nr
FROM fine AS f1
WHERE UNIQUE
  (SELECT f2.player_nr
   FROM fine AS f2
   WHERE f2.player_nr = f1.player_nr);
```
- Not implemented in PostgreSQL, can be replaced by HAVING

Slide 177

177

OVERLAPS OPERATOR

- Checks whether two time periods overlap. Returns **TRUE** if the two periods share **any** common moment in time.
- Question: List the players, including their names, board positions, start date and end date, who served from 1 January 1991 to 31 December 1993

```
SELECT player_nr, role, start_date, end_date
FROM board_member
WHERE (start_date, end_date) OVERLAPS ('1991-01-01', '1993-12-31');
```

Slide 178

178

COMBINATION EXERCISE 1

- Question: List the customers who went on a journey that also included the customer with customer number 126. Show the customer numbers.

Slide 179

179

COMBINATION EXERCISE 1 SOLUTION

- Question: List the clients who went on a journey that also included the client with client number 126. Show the customer numbers.

```
SELECT c.client_nr
FROM client c
INNER JOIN participation p USING (client_nr)
WHERE EXISTS
  (SELECT *
   FROM participation AS otherparticipations
   WHERE client_nr = 126 AND otherparticipations.journey_nr =
     p.journey_nr)
GROUP BY c.client_nr;
```

Slide 180

180

COMBINATION EXERCISE 2

- Question: Show all celestial object data for planets visited on journeys without client 126.

Slide 181

181

COMBINATION EXERCISE 2 SOLUTION

- Question: Show all celestial object data for planets visited on journeys without client 126.

```
SELECT *
FROM celestial_object c
WHERE EXISTS
  (SELECT *
   FROM visit v
   WHERE NOT EXISTS
     (SELECT *
      FROM participation p
      WHERE client_nr = 126 AND p.journey_nr = v.journey_nr)
   AND c.objectname = v.objectname)
AND c.satellite_of = 'Sun';
```

Slide 182

182

CHALLENGE: WHAT DOES THIS QUERY DO ?

```
SELECT player_nr
FROM player AS s
WHERE NOT EXISTS
  (SELECT *
   FROM match AS w1
   WHERE player_nr = 57
   AND NOT EXISTS
     (SELECT *
      FROM match AS w2
      WHERE w1.team_nr = w2.team_nr
      AND s.player_nr = w2.player_nr))
AND player_nr NOT IN
  (SELECT player_nr
   FROM match
   WHERE team_nr IN
     (SELECT team_nr
      FROM team
      WHERE team_nr NOT IN
        (SELECT team_nr
         FROM match
         WHERE player_nr= 57)));
```

Slide 183

183

OPO: Data Management

Aimée Lynn Backiel
Wim Bertels
Alain Van Der Jeught

Module B: Advanced SQL Non-correlated subqueries

TI / BCS

Attribution-NonCommercial-ShareAlike 4.0 Unported Licence

2025-2026

184

SYMBOLS USED IN THIS COURSE

Symbol	Category	Typical Slide Use
	Positioning	Positioning, purpose
	Objective	Objective, goal
	Attention	Warnings, key pitfalls, critical notes
	Theory	Concept explanations, definitions
	Practice	Exercises, hands-on labs
	Clip	Demo, Clip
	Poll	Live questions / Quiz
	Informational	Neutral notes, references, reading tips. Not examinable

Slide 185

185

PURPOSE

- Introduce** the concept of **non-correlated subqueries** and their role in SQL querying.
- Show** how they retrieve data **independently** from the outer query.
- Demonstrate** when and why they are preferred for clarity and performance.

Slide 186

186

OBJECTIVES

- **Define** a non-correlated subquery.
- **Identify** scenarios where non-correlated subqueries are appropriate.
- **Write** SQL statements using non-correlated subqueries in SELECT, FROM, and WHERE clauses.
- **Interpret** query execution flow to understand how the database processes these subqueries.

Slide 187

187

WHAT IS A SUBQUERY?

- A **query inside another query**.
- The inner query runs first, and its **result is passed** to the outer query.
- Can be placed in the WHERE, FROM, or SELECT clauses.
- Subqueries can be **nested** inside other subqueries to multiple levels.
- **Outer query** = the main question you're asking.
- **Inner query (subquery)** = a smaller question whose answer helps answer the main question.

Slide 188

188

WHY DO WE USE SUBQUERIES?

- Break a complex query into **smaller, simpler problems** whose results can be used in the main query, e.g.
- Similar to programming: break down a complex method into **smaller, atomic tasks**.
- Makes SQL easier to **read, write, and maintain**.
- **Filter based on calculated results**.
- **Reuse logic** in multiple clauses.
- **Avoid the need for storing intermediate results** (temporary tables).

Slide 189

189

NON-CORRELATED VS CORRELATED SUBQUERIES

- **1. Non-Correlated Subquery (scope of this presentation)**
 - **Independent** of the outer query — it can be executed **on its own**.
 - The subquery runs **once**, and its result is then used by the outer query.
 - **Does not** refer to any column from the outer query.
- **2. Correlated Subquery (see correlated subqueries)**
 - **Depends on** values from the outer query - it **cannot** be executed **on its own**.
 - The subquery is re-run **for each row** processed by the outer query.
 - References columns from the outer query in its condition.

Slide 190

190

TYPES OF SUBQUERIES

- **Scalar** subquery: output = 1 row, 1 column (i.e. 1 value)
- **Row** subquery: output = 1 row
- **Column** subquery: output = multiple rows with 1 value each
- **Table** subquery: output = multiple rows and columns

Slide 191

191

SCALAR SUBQUERY (1 ROW, 1 COLUMN)

- Example:
For each planet, calculate and display the difference in size compared to the Sun.

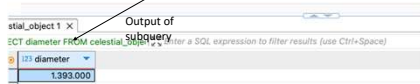
Slide 192

192

SCALAR SUBQUERY (1 ROW, 1 COLUMN)

- Example:
For each planet, calculate and display the difference in size compared to the Sun.

```
=SELECT objectname, diameter - (SELECT diameter  
FROM celestial_object  
WHERE objectname = 'Sun') AS difference  
FROM celestial_object  
WHERE satellite_of = 'Sun';
```



objectname	diameter
Sun	1,393,000

Slide 193

193

SCALAR SUBQUERY (1 ROW, 1 COLUMN)

```
SELECT objectname, diameter - (SELECT diameter  
FROM celestial_object  
WHERE objectname = 'Sun') AS difference  
FROM celestial_object  
WHERE satellite_of = 'Sun';
```

Slide 194

194

SCALAR SUBQUERY (1 ROW, 1 COLUMN)

- Example 2:
- List the celestial bodies with a diameter larger than Venus.

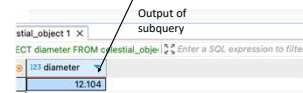
Slide 195

195

SCALAR SUBQUERY (1 ROW, 1 COLUMN)

- Example 2:
- List the celestial bodies with a diameter larger than Venus.

```
=SELECT objectname, diameter  
FROM celestial_object  
WHERE diameter >  
(SELECT diameter  
FROM celestial_object  
WHERE objectname = 'Venus');
```



objectname	diameter
Jupiter	12,104

Slide 196

196

SCALAR SUBQUERY (1 ROW, 1 COLUMN)

```
SELECT objectname, diameter  
FROM celestial_object  
WHERE diameter >  
(SELECT diameter  
FROM celestial_object  
WHERE objectname = 'Venus');
```

Slide 197

197

ROW SUBQUERY (1 ROW)

- Example:
- List all players with the gender and hometown as the player with number 7.

Slide 198

198

ROW SUBQUERY (1 ROW)

- Example:
- List all players with the same gender and hometown as the player with number 7.

```
SELECT player_nr, last_name, city
FROM player
WHERE (city, gender) =
  (SELECT city, gender
   FROM player
   WHERE player_nr = 7);
```

Output of subquery

CT city, gender FROM player
Den Haag M

Slide 199

199

ROW SUBQUERY (1 ROW)

```
SELECT player_nr, last_name, city
FROM player
WHERE (lcity, gender) =
  (SELECT city, gender
   FROM player
   WHERE player_nr = 7);
```

Slide 200

200

COLUMN SUBQUERY (MULTIPLE ROWS, 1 VALUE EACH)

- Example:
List all moons.

Slide 201

201

COLUMN SUBQUERY (MULTIPLE ROWS, 1 VALUE EACH)

- Example:
List all moons
in the solarsystem.

```
SELECT objectname
FROM celestial_object
WHERE satellite_of IN
  (SELECT objectname
   FROM celestial_object
   WHERE satellite_of = 'Sun');
```

Output of subquery

objectname
Earth
Pluto
Mercurius
Jupiter
Uranus
Venus
Mars
Saturn
Neptune

Slide 202

202

COLUMN SUBQUERY (MULTIPLE ROWS, 1 VALUE EACH)

```
SELECT objectname
FROM celestial_object
WHERE satellite_of IN
  (SELECT objectname
   FROM celestial_object
   WHERE satellite_of = 'Sun');
```

Slide 203

203

TABLE SUBQUERY (MULTIPLE ROWS AND COLUMNS)

- Produces a temporary result set.
- If the subquery appears in the FROM clause of the main query, it must be assigned an alias.

Slide 204

204

TABLE SUBQUERY (MULTIPLE ROWS AND COLUMNS)

- Example:
List the journeys by departure date that visit any celestial object visited at least five times in total across all journeys.

Slide 205

205

TABLE SUBQUERY (MULTIPLE ROWS AND COLUMNS)

- Example:
List the journeys by departure date that visit any celestial object visited at least five times in total across all journeys.
- Intermediate step: list the objects that were visited at least five times across all journeys
SELECT objectname
FROM visit
GROUP BY objectname
HAVING COUNT(*) >=5;

Slide 206

206

TABLE SUBQUERY (MULTIPLE ROWS AND COLUMNS)

- Example: List the journeys by departure date that visit any celestial object visited at least five times in total across all journeys.

```
SELECT j.journey_nr, departure_date
FROM journey j
INNER JOIN visit v ON j.journey_nr = v.journey_nr
INNER JOIN (
  SELECT objectname
  FROM visit
  GROUP BY objectname
  HAVING COUNT(*) >=5
) AS oftenvisited ON v.objectname = oftenvisited.objectname
GROUP BY j.journey_nr, departure_date;
```

SQL Error: ORA-00979: not a GROUP BY expression

Slide 207

207

TABLE SUBQUERY (MULTIPLE ROWS AND COLUMNS)

```
SELECT j.journey_nr, departure_date
FROM journey j
INNER JOIN visit v ON j.journey_nr = v.journey_nr
INNER JOIN (
  SELECT objectname
  FROM visit
  GROUP BY objectname
  HAVING COUNT(*) >=5
) AS oftenvisited ON v.objectname = oftenvisited.objectname
GROUP BY j.journey_nr, departure_date;
```

Slide 208

208

ANOTHER SMALL THING

- Question: List all journeys that have not visited the moon.

Slide 209

209

ANOTHER SMALL THING

- Question: List all journeys that have not visited the moon.

```
SELECT j.journey_nr
FROM journey j
INNER JOIN visit v ON j.journey_nr = v.journey_nr
WHERE v.objectname <> 'Moon'
GROUP BY j.journey_nr;
```

Is this query a correct answer to the question?

Slide 210

210

SOLUTION (INTERMEDIATE STEP)

- List all journeys that did visit the moon

```
SELECT j.journey_nr
FROM journey j
INNER JOIN visit v ON j.journey_nr = v.journey_nr
WHERE v.objectname = 'Moon'
GROUP BY j.journey_nr;
```



Slide 211

211

SOLUTION

- Next, list all journeys that do not appear in the resultset of the subquery:

```
SELECT journey_nr
FROM journey
WHERE journey_nr NOT IN
  (SELECT j.journey_nr
   FROM journey j
   INNER JOIN visit v ON j.journey_nr = v.journey_nr
   WHERE v.objectname = 'Moon'
   GROUP BY j.journey_nr);
```



Slide 212

212

SOLUTION

- Solution simplified

```
SELECT journey_nr
FROM journey
WHERE journey_nr NOT IN
  (SELECT journey_nr
   FROM visit
   WHERE objectname = 'Moon');
```



Slide 213

213

CHALLENGE

- Try writing this question without a subquery (on a rainy day..., if you really have too much time on your hands...):

```
SELECT AVG(total)
FROM
  (SELECT player_nr, SUM(amount) AS total
   FROM fine
   GROUP BY player_nr) AS totals
```

- What does this query show?



Slide 214

214



OPO: Data Management

Aimée Lynn Backiel
Wim Bertels
Alain Van Der Jeught

Module B: Advanced SQL Common Table Expressions (CTE)

TI / BCS

Attribution-NonCommercial-ShareAlike 4.0 Unported Licence

2025-2026

215

SYMBOLS USED IN THIS COURSE

Symbol	Category	Typical Slide Use
	Positioning	Positioning, purpose
	Objective	Objective, goal
	Attention	Warnings, key pitfalls, critical notes
	Theory	Concept explanations, definitions
	Practice	Exercises, hands-on labs
	Clip	Demo, Clip
	Poll	Live questions / Quiz
	Informational	Neutral notes, references, reading tips. Not examinable

Slide 216

216

PURPOSE

- To **simplify complex queries** by breaking them into modular, readable parts.
- To allow **query reuse** within a single SQL statement.
- To make **recursive queries** (like hierarchical data, graphs, trees) possible.
- To improve **maintainability and clarity** of SQL code.
- To serve as a **step-by-step build-up** for analysis and transformations.

Slide 217

217

OBJECTIVES

- **Define and use CTEs** with WITH ... AS (...).
- **Refactor complex SQL** into smaller, logical pieces using CTEs.
- **Compare CTEs with subqueries** (understand readability and performance trade-offs).
- **Use recursive CTEs** to handle hierarchical/graph structures (e.g., org charts, bill of materials).
- **Apply CTEs in real scenarios** for analytics, reporting, and stepwise query building.

Slide 218

218

WHAT ARE CTEs?

- A **Common Table Expression (CTE)** is a **temporary, named result set defined within a query using the WITH clause**.
- Can be referenced from other statements (Select, Insert, Update, or Delete)
- Only exists while query is executed (not stored in DB!)
- Must be executed again every time
- They can simplify complex joins and subqueries
- And extra functionalities too
- Recursive CTEs can loop

Slide 219

219

CTE SYNTAX

```
WITH cte_name (column_list) AS (  
    CTE_query_definition  
)  
statements;
```

- Specify the name of the CTE with optional column list
- Inside the WITH body, specify a query that returns a result set. If no column list was specified, columns will be based on the SELECT
- Use the CTE like a table or view in the statement which can be a SELECT, INSERT, UPDATE, or DELETE.

Slide 220

220

COMPARISON WITH SUBQUERY

```
SELECT *  
FROM (SELECT *  
      FROM player)  
AS player_sub;
```

Subqueries like
we've been using
for a long time

```
WITH player_sub as (  
    SELECT *  
    FROM player  
)  
SELECT *  
FROM player_sub;
```

CTE equivalent

Slide 221

221

SIMPLE CTE EXAMPLE

```
WITH cte_film AS (  
    SELECT film_id, title,  
           (CASE  
            WHEN length < 30 THEN 'Short'  
            WHEN length < 90 THEN 'Medium'  
            ELSE 'Long'  
            END) length  
    FROM film  
)  
SELECT film_id, title, length  
FROM cte_film  
WHERE length = 'Long'  
ORDER BY title;
```

Defines CTE cte_film

SELECT statement run on cte_film

Slide 222

222

SIMPLE CTE EXAMPLE

```
WITH cte_film AS (
  SELECT film_id, title,
    (CASE
      WHEN length < 30 THEN 'Short'
      WHEN length < 90 THEN 'Medium'
      ELSE 'Long'
    END) length
  FROM film
)
SELECT film_id, title, length
FROM cte_film
WHERE length = 'Long'
ORDER BY title;
```

film_id	title	length
4	Affair Prejudice	Long
11	Alamo Videotape	Long
19	Amadeus Holy	Long

Slide 223

223

CTE TABLE JOIN EXAMPLE

```
WITH cte_rental AS (
  SELECT staff_id,
    COUNT(rental_id) rental_count
  FROM rental
  GROUP BY staff_id
)
SELECT s.staff_id,
  first_name,
  last_name,
  rental_count
FROM staff s
  INNER JOIN cte_rental USING (staff_id);
```

Defines CTE to create result set with staff id and number of rentals

Join staff table with CTE using staff_id column

Slide 224

224

CTE TABLE JOIN EXAMPLE

```
WITH cte_rental AS (
  SELECT staff_id,
    COUNT(rental_id) rental_count
  FROM rental
  GROUP BY staff_id
)
SELECT s.staff_id,
  first_name,
  last_name,
  rental_count
FROM staff s
  INNER JOIN cte_rental USING (staff_id);
```

staff_id	first_name	last_name	rental_count
1	Mike	Hillyer	8040
2	Jon	Stephens	8004

Slide 225

225

CTE FAMILY TREE EXAMPLE

```
CREATE TABLE familytree(
  name varchar(16) NOT NULL,
  mother varchar(16),
  father varchar(16),
  CONSTRAINT familytree_pk
    PRIMARY KEY (name),
  CONSTRAINT mother_fk
    FOREIGN KEY (mother)
    REFERENCES familytree(name)
);
```

Slide 226

226

CTE FAMILY TREE EXAMPLE

```
INSERT INTO familytree VALUES
('top_of_tree', NULL, NULL),
('out', NULL, NULL),
('grandfather', 'top_of_tree', 'top_of_tree'),
('grandmother', 'top_of_tree', 'top_of_tree'),
('oma', 'top_of_tree', 'top_of_tree'),
('opa', 'top_of_tree', 'opa_dad'),
('father', 'oma', 'opa'),
('uncle', 'oma', 'opa'),
('aunt', 'grandmother', 'grandfather'),
('mother', 'grandmother', 'grandfather'),
('cousin', 'out', 'uncle'),
('brother', 'mother', 'father'),
('child', 'mother', 'father');
```



Slide 227

227

CTE FAMILY TREE EXAMPLE

Find all the children of "opa"

```
SELECT *
FROM familytree
WHERE father='opa';
```

name	mother	father
father	oma	opa
uncle	oma	opa

(2 rows)

Slide 228

228

CTE FAMILY TREE EXAMPLE

Find all the children and grandchildren of "opa"

```
SELECT *
FROM familytree
WHERE father = 'opa'
OR father IN
(SELECT name
FROM familytree
WHERE father = 'opa');
```

name	mother	father
father	oma	opa
uncle	oma	opa
child	mother	father
brother	mother	father
cousin	out	uncle

(5 rows)

Slide 229

229

CTE FAMILY TREE EXAMPLE

- What if we want to see the great-grandchildren?
- How many generations are in the tree?
- Where do we stop?

Slide 230

230

RECURSIVE CTE

- Recursive queries refer to recursive CTEs
- Use for querying hierarchical data
- Syntax:

```
WITH RECURSIVE cte_name AS (
  CTE_query_definition --non-recursive
  UNION [ALL]
  CTE_query_definition --recursive
)
SELECT * FROM cte_name;
```

Slide 231

231

RECURSIVE CTE SYNTAX

```
WITH RECURSIVE cte_name AS (
  CTE_query_definition --non-recursive
  UNION [ALL]
  CTE_query_definition --recursive
)
SELECT * FROM cte_name;
```

- Elements of a recursive CTE
 - Non-recursive term: forms the base result set
 - Recursive term: references the CTE name, joined with UNION or UNION ALL
 - Termination check: recursion stops when no rows are returned from the previous iteration

Slide 232

232

RECURSIVE CTE EXAMPLE

```
CREATE TABLE employees (
  employee_id serial PRIMARY KEY,
  full_name VARCHAR NOT NULL,
  manager_id INT
);
```

--hierarchical data: manager is an employee, manager_id is employee_id of employee's manager

```
INSERT INTO employees (
  employee_id, full_name, manager_id
) VALUES
(1, 'Michael North', NULL),
(2, 'Megan Berry', 1),
(3, 'Sarah Berry', 1),
(4, 'Zoe Black', 1),
(5, 'Tim James', 1),
(6, 'Bella Tucker', 2),
(7, 'Ryan Metcalfe', 2),
(8, 'Max Mills', 2),
(9, 'Benjamin Glover', 2),
(10, 'Carolyn Henderson', 3),
(11, 'Nicola Kelly', 3),
(12, 'Alexandra Climo', 3),
(13, 'Dominic King', 3),
(14, 'Leonard Gray', 4),
(15, 'Eric Rampling', 4),
(16, 'Piers Paige', 7),
(17, 'Ryan Henderson', 7),
(18, 'Frank Tucker', 8),
(19, 'Nathan Ferguson', 8),
(20, 'Kevin Rampling', 8);
```

Slide 233

233

RECURSIVE CTE EXAMPLE

```
WITH RECURSIVE subordinates AS (
  SELECT employee_id, manager_id, full_name
  FROM employees
  WHERE employee_id = 2
  UNION
  SELECT e.employee_id, e.manager_id, e.full_name
  FROM employees e
  INNER JOIN subordinates s ON s.employee_id = e.manager_id
)
SELECT *
FROM subordinates;
```

CTE

Non-recursive term

Recursive term

statement

Slide 234

234

RECURSIVE CTE EXECUTION

1. Non-recursive term returns R₀

employee_id	manager_id	full_name
2	1	Megan Berry

2. The recursive term returns the subordinates of employee id 2 (Megan Berry)

employee_id	manager_id	full_name
6	2	Bella Tucker
7	2	Ryan Metcalfe
8	2	Max Mills
9	2	Benjamin Glover

Slide 235

235

RECURSIVE CTE EXECUTION

3. Recursive term repeats, returns subordinates of those subordinates

employee_id	manager_id	full_name
16	7	Piers Paige
17	7	Ryan Henderson
18	8	Frank Tucker
19	8	Nathan Ferguson
20	8	Kevin Rampling

4. The third iteration returns an empty set, no subordinates of 16,17,18,19, or 20

5. The union of all result sets is returned

Slide 236

236

RECURSIVE CTE FAMILY TREE EXAMPLE

```
WITH RECURSIVE child_of(name, mother, father) AS (
    SELECT name, mother, father
    FROM familytree
    WHERE father = 'opa_dad'
    UNION ALL
    SELECT f.name, f.father, f.mother
    FROM familytree f, child_of c
    WHERE f.father = c.name
)
SELECT *
FROM child_of;
```

Slide 237

237

RECURSIVE CTE FAMILY TREE EXAMPLE

name	mother	father
opa	top_of_tree	opa_dad
father	oma	opa
uncle	oma	opa
child	mother	father
brother	mother	father
cousin	out	uncle

(6 rows)

Slide 238

238

CTE FAMILY TREE EXAMPLE

Show the tree branch?

```
WITH RECURSIVE child_of(name, mother, father, branch) AS (
    SELECT name, mother, father, 'opa_dad'
    FROM familytree
    WHERE father = 'opa_dad'
    UNION ALL
    SELECT f.name, f.mother, f.father, branch || ' - ' || f.father
    FROM familytree f, child_of c
    WHERE f.father = c.name
)
SELECT name, branch
FROM child_of;
```

Slide 239

239

RECURSIVE CTE FAMILY TREE EXAMPLE

name	branch
opa	opa_dad
father	opa_dad - opa
uncle	opa_dad - opa
child	opa_dad - opa - father
brother	opa_dad - opa - father
cousin	opa_dad - opa - uncle

(6 rows)

Slide 240

240

INFINITE LOOPING

- No automatic checking for infinite loops
- Include checks in your query
 - Use a counter
 - Use loop detection
 - Combine both

Slide 241

241

USING A COUNTER

```
WITH RECURSIVE child_of(name, mother, father, depth) AS (
    SELECT name, mother, father, 1
    FROM familytree
    WHERE father = 'opa'
    UNION ALL
    SELECT f.name, f.mother, f.father, depth + 1
    FROM familytree f, child_of c
    WHERE f.father = c.name
    AND c.depth < 7
)
SELECT *
FROM child_of;
```

Slide 242

242

USING A COUNTER

name	mother	father	depth
father	oma	opa	1
uncle	oma	opa	1
child	mother	father	2
brother	mother	father	2
cousin	out	uncle	2

(5 rows)

Slide 243

243

USING LOOP DETECTION

```
INSERT INTO familytree VALUES
('mummy', NULL, NULL),
('grammy', NULL, NULL),
('cringe', 'mummy', 'daddy'),
('daddy', 'grammy', 'grampy'),
('grampy', 'grammy', 'cringe');
```

Slide 244

244

USING LOOP DETECTION

```
WITH RECURSIVE child_of(name, mother, father, path, loop) AS (
    SELECT name, mother, father, ARRAY[father] as path, false
    FROM familytree
    WHERE father = 'grampy'
    UNION ALL
    SELECT f.name, f.mother, f.father,
    CAST(c.path || ARRAY[f.father] as varchar(16)[]) as path,
    f.father = ANY(path)
    FROM familytree f, child_of c
    WHERE f.father = c.name
    AND NOT loop
)
SELECT *
FROM child_of;
```

Slide 245

245

USING LOOP DETECTION

name	mother	father	path	loop
daddy	grammy	grampy	{grampy}	f
cringe	mummy	daddy	{grampy,daddy}	f
grampy	grammy	cringe	{grampy,daddy, cringe}	f
daddy	grammy	grampy	{grampy,daddy,grampy}	t

(4 rows)

Slide 246

246

CTE: INSERT, UPDATE, AND DELETE

- Postgresql-specific allows CTEs in INSERT, UPDATE, and DELETE

```
CREATE TEMPORARY TABLE demo (x NUMERIC);
INSERT INTO demo
VALUES (random()), (random()), (random()) RETURNING x;
```

x
0.0342996246181428
0.268639853224158
0.422058953437954

(3 rows)

Slide 247

247

CTE: INSERT, UPDATE, AND DELETE

- Postgresql-specific allows CTEs in INSERT, UPDATE, and DELETE

```
WITH source AS (
  INSERT INTO demo
  VALUES (random()), (random()), (random()) RETURNING x
)
SELECT AVG(x) FROM source;
```

avg
0.5563394334167243333

(1 row)

Slide 248

248

CTE: INSERT, UPDATE, AND DELETE

```
WITH RECURSIVE child_of(name) AS (
  SELECT 'father'::varchar
  UNION ALL
  SELECT f.name
  FROM child_of c JOIN familytree f
  ON (c.name=f.father)
)
DELETE FROM familytree
USING child_of
WHERE child_of.name = familytree.name;

DELETE 7
```

Slide 249

249

DEBUGGING WITH CTE

- Find all players with a name of the same length (in steps)

```
WITH length_names AS (
  SELECT last_name, length(last_name) AS n
  FROM player),
equally_long_names AS (
  SELECT l1.last_name AS name1, l2.last_name AS name2
  FROM length_names l1 JOIN length_names l2
  ON l1.n = l2.n
  WHERE l1.last_name <> l2.last_name),
overview_list AS (
  SELECT name1, array_agg(name2) AS list
  FROM equally_long_names
  GROUP BY name1)
SELECT l.last_name, o.list
FROM length_names l LEFT JOIN overview_list o
ON l.last_name = o.name1;
```

Slide 250

250

DEBUGGING WITH CTES

last_name	list
Elfring	{"Moerman ", "Hofland "}
Permentier	{"Bakker, de ", "Niewenburg ", "Bakker, de ", "Niewenburg "}
Wijers	
Niewenburg	{"Permentier ", "Bakker, de ", "Permentier "}
Cools	
Cools	
Bischoff	{"Meuleman "}
Bakker, de	{"Permentier ", "Niewenburg ", "Permentier "}
Bohemien, van	
Hofland	{"Moerman ", "Elfring "}
Meuleman	{"Bischoff "}
Permentier	{"Bakker, de ", "Niewenburg ", "Bakker, de ", "Niewenburg "}
Moerman	
Baalen, van	{"Hofland ", "Elfring "}

(14 rows)

Notice anything???

Slide 251

251

DEBUGGING WITH CTES

```
CREATE TEMPORARY TABLE debug_table -- store intermediate results from the CTEs
(id serial, t text, r text); -- to try to find any errors
WITH length_names AS (
  SELECT last_name, length(last_name) AS n
  FROM player),
debug_length_names AS (
  INSERT INTO debug_table(t,r)
  SELECT 'length_names', ROW(1,*)::text -- store results
  FROM length_names l),
equally_long_names AS (
  SELECT l1.last_name AS name1, l2.last_name AS name2
  FROM length_names l1 JOIN length_names l2
  ON l1.n = l2.n
  WHERE l1.last_name <> l2.last_name),
overview_list AS (
  SELECT name1, array_agg(name2) AS list
  FROM equally_long_names
  GROUP BY name1)
SELECT l.last_name, o.list
FROM length_names l LEFT JOIN overview_list o
ON l.last_name = o.name1;
```

Slide 252

252

DEBUGGING WITH CTE

last_name	list
Elfring	{"Moerman ", "Hofland "}
Permentier	{"Bakker, de ", "Niewenburg ", "Bakker, de ", "Niewenburg "}
Wijers	
Niewenburg	{"Permentier ", "Bakker, de ", "Permentier "}
Cools	
Bischoff	{"Meuleman "}
Bakker, de	{"Permentier ", "Niewenburg ", "Permentier "}
Bohemen, van	
Hofland	{"Moerman ", "Elfring "}
Meuleman	{"Bischoff "}
Permentier	{"Bakker, de ", "Niewenburg ", "Bakker, de ", "Niewenburg "}
Moerman	{"Hofland ", "Elfring "}
Baalen, van	
(14 rows)	

253

DEBUGGING WITH CTE

SELECT * FROM debug_table;

id	t	r
1	length_names	("Elfring ", 7)
2	length_names	("Permentier ", 10)
3	length_names	("Wijers ", 6)
4	length_names	("Niewenburg ", 10)
5	length_names	("Cools ", 5)
6	length_names	("Cools ", 5)
7	length_names	("Bischoff ", 8)
8	length_names	("Bakker, de ", 10)
9	length_names	("Bohemen, van ", 12)
10	length_names	("Hofland ", 7)
11	length_names	("Meuleman ", 8)
12	length_names	("Permentier ", 10)
13	length_names	("Moerman ", 7)
14	length_names	("Baalen, van ", 11)
(14 rows)		

254



OPO: Data Management

Aimée Lynn Backiel
Wim Bertels
Alain Van Der Jeught

Module B: Advanced SQL

Window functions

TI / BCS Attribution-NonCommercial-ShareAlike 4.0 Unported Licence 2025-2026

255

SYMBOLS USED IN THIS COURSE

Symbol	Category	Typical Slide Use
	Positioning	Positioning, purpose
	Objective	Objective, goal
	Attention	Warnings, key pitfalls, critical notes
	Theory	Concept explanations, definitions
	Practice	Exercises, hands-on labs
	Clip	Demo, Clip
	Poll	Live questions / Quiz
	Informational	Neutral notes, references, reading tips. Not examinable

256

PURPOSE

- Introduce **window (analytic) functions** as a powerful SQL feature for advanced analysis.
- Show how they extend beyond aggregates by allowing calculations **over partitions of rows** while preserving detail rows.
- Write queries that perform **ranking, running totals, moving averages, and comparisons** without complex subqueries.

257

OBJECTIVES

- Define** what a window function is and distinguish it from regular aggregate functions.
- Use OVER()** to apply functions across partitions and orderings.
- Write queries** with common window functions:
 - Ranking: ROW_NUMBER(), RANK(), DENSE_RANK()
 - Value access: LAG(), LEAD(), FIRST_VALUE(), LAST_VALUE()
 - Aggregations: SUM(), AVG(), etc. **with OVER()**
- Explain and apply** PARTITION BY and ORDER BY inside window definitions.

258

DIFFERENT PERSPECTIVE ON 2 CONCEPTS

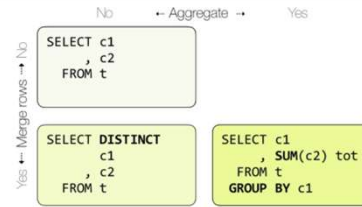
- **Merging** rows based on a property
 - GROUP BY : more options
 - DISTINCT : remove duplicate rows
- **Aggregating** data from related rows
 - GROUP BY needed to form the groups
 - Aggregation functions, e.g. count, sum

Slide 259

259

OVER (PARTITION BY)

The Problem

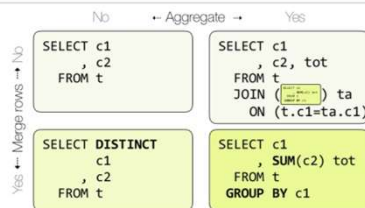


Slide 260

260

OVER (PARTITION BY)

The Problem

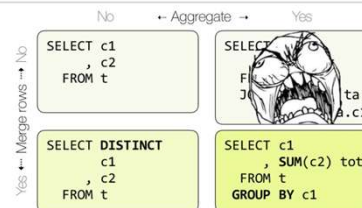


Slide 261

261

OVER (PARTITION BY)

The Problem

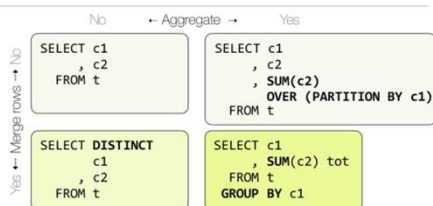


Slide 262

262

OVER (PARTITION BY)

Since SQL:2003



Slide 263

263

WINDOW FUNCTION

- A **window function** performs a calculation across a set of rows that are **related to the current row** (called the *window* or *partition*).
- Unlike GROUP BY aggregates, it **does not collapse rows** — each row stays visible, but gets extra calculated values.
- Syntax always uses **OVER(...)**.

Slide 264

264

WINDOW FUNCTION COMPONENTS

- **Function**
 - An aggregate function: SUM(), AVG(), MIN(), MAX(), COUNT().
 - A ranking function: ROW_NUMBER(), RANK(), DENSE_RANK().
 - A value function: LAG(), LEAD(), FIRST_VALUE(), LAST_VALUE().
- **OVER(...)**
 - Required keyword → tells SQL you're using a **windowed version** of the function.
- **PARTITION BY** (optional)
 - Splits the data into subgroups ("windows") for independent calculation.
 - Similar to GROUP BY, but rows are not collapsed.
- **ORDER BY** (optional)
 - Defines the order of rows **within each partition** (important for ranking, running totals, lag/lead).
- **Frame clause** (optional)
 - Defines exactly **which rows in the partition** are visible to the function.
 - Default for most aggregates = RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW.

Slide 265

265

WINDOW FUNCTIONS USAGE

- Running totals
- Moving averages
- Ranking (leaderboards)
- Previous/next row comparisons (with LAG / LEAD)
- Percentiles and advanced analytics

Slide 266

266

FUNCTIONS - ROW_NUMBER

- Add a row number

```
SELECT row_number() OVER(), player_nr
FROM tennis_en.player;
```

- --are these row numbers fixed?

row_number	player_nr
1	2
2	6
3	7
4	39
5	44
6	57
7	83
8	95
9	100
10	8
11	27
12	28
13	104
14	112

Slide 267

267

FUNCTIONS - ROW_NUMBER

- Add a row number

```
SELECT row_number() OVER(), player_nr
FROM tennis_en.player
ORDER BY 2;
```

- --with 'fixed' row numbers ?
- --ORDER BY comes after SELECT

row_number	player_nr
1	2
2	6
3	7
10	8
11	27
12	28
4	39
5	44
6	57
7	83
8	95
9	100
13	104
14	112

Slide 268

268

ORDER BY

- Manipulate sequence of row numbers

```
SELECT row_number() OVER(ORDER BY player_nr), player_nr
FROM tennis_en.player;
```

- Row numbering without an explicit order is non-deterministic.
- Always specify ORDER BY if you need reproducible and stable row numbers.

row_number	player_nr
1	2
2	6
3	7
6	28
7	39
8	44
9	57
10	83
11	95
12	100
13	104
14	112

Slide 269

269

ORDER BY

```
SELECT row_number() OVER(ORDER BY player_nr,
player_nr, city, association_nr
FROM tennis_en.player
ORDER BY association_nr NULLS FIRST;
```

Execution sequence

1. FROM → read rows from tennis_en.player.
2. SELECT list → compute ROW_NUMBER() OVER(ORDER BY player_nr) for each row.
At this step, the DB knows how to number rows based on player_nr.
3. ORDER BY (outer) → once all rows have their row numbers, the final result is sorted by association_nr NULLS FIRST.

row_number	player_nr	city	association_nr
6	28	Leiden	NULL
11	95	Voorburg	NULL
3	7	Den Haag	NULL
7	39	Den Haag	NULL
8	44	Rijswijk	1124
14	112	Rotterdam	1519
10	83	Den Haag	1608
1	2	Den Haag	2411
5	27	Zoetermeer	2513
4	8	Rijswijk	2983
9	57	Den Haag	4409
12	100	Den Haag	6504
13	104	Zoetermeer	7060
2	6	Den Haag	8467

Slide 270

270

FUNCTIONS – RANK()

- RANK() is a **window function** that assigns a **ranking number** to rows within a partition, **based on the order you specify**.
 - If two or more rows are **tied** (same ordering value), they receive the **same rank**.
 - But the next rank is **skipped** accordingly
- Example:


```
SELECT rank() OVER(ORDER BY city), city
FROM tennis_en.player
ORDER BY city NULLS FIRST;
```

rank	city
1	Den Haag
1	Den Haag
1	Den Haag
1	Den Haag
1	Den Haag
1	Den Haag
1	Den Haag
8	Leiden
9	Rijswijk
9	Rijswijk
11	Rotterdam
12	Voorburg
13	Zoetermeer
13	Zoetermeer

Slide 271

271

FUNCTIONS – RANK()

- Pseudo code from example in previous slide
- Rows sorted by city
- Running through the rows
- For each row:
 - IF the current row is the first row in the partition - indicate 1
 - ELSE IF city is equal to previous city - indicate previous rank
 - ELSE indicate the count of rows so far

Slide 272

272

FUNCTIONS - DENSE_RANK()

- DENSE_RANK() is a **window function** that assigns a **ranking number** to rows within a partition, **based on the order you specify**.
 - Ties get the same rank, just like RANK().
 - But unlike RANK(), **no gaps are left** in the sequence when ties occur.
- Example


```
SELECT dense_rank() OVER(ORDER BY city), city
FROM tennis_en.player
ORDER BY city NULLS FIRST;
```

dense_rank	city
1	Den Haag
1	Den Haag
1	Den Haag
1	Den Haag
1	Den Haag
1	Den Haag
1	Den Haag
2	Leiden
3	Rijswijk
3	Rijswijk
4	Rotterdam
5	Voorburg
6	Zoetermeer
6	Zoetermeer

Slide 273

273

FUNCTIONS - AGGREGATES

- SUM() used as a **window function** (with OVER(...)) can compute a **cumulative sum (running total)** if you include an ORDER BY clause in the window.
- ORDER BY determines the sequence of accumulation
- Example:


```
SELECT date, SUM(amount) OVER(ORDER BY date)
FROM tennis_en.fine
ORDER BY date;
```

date	sum
2025-03-03	1.011
2025-03-03	1.011
2025-03-03	1.011
2025-03-03	1.011
2025-04-12	1.136
2025-05-23	1.311
2025-06-12	1.561
2025-06-13	2.111
2025-06-13	2.111

Slide 274

274

PARTITION BY

- In window functions, a **partition** is the **subset of rows** in the result set **over which the window function is applied**.
- It's similar to GROUP BY in that it divides rows into groups, but different because:
 - GROUP BY collapses each group into one row.
 - PARTITION BY (in OVER(...)) **keeps all the detail rows**, but **calculates the window function within each partition**.
- Important:
 - Any aggregate (SUM, AVG, ...) can be used as a window function.
 - But not all window functions are aggregates (ROW_NUMBER(), RANK(), LAG(), ...).

Slide 275

275

PARTITION BY

- Add a row number per city and sort result by city.

```
SELECT row_number() OVER(PARTITION BY city), city, player_nr
FROM tennis_en.player
ORDER BY city;
```

row_number	city	player_nr
1	Den Haag	2
2	Den Haag	6
3	Den Haag	7
4	Den Haag	39
5	Den Haag	100
6	Den Haag	67
7	Den Haag	83
1	Leiden	28
1	Rijswijk	44
2	Rijswijk	8
1	Rotterdam	112
1	Voorburg	95
1	Zoetermeer	27
2	Zoetermeer	104

Meaning of row number?

Slide 276

276

PARTITION BY WITH ORDER BY

- Add a row number per city with fixed order based on pk player_nr. Sort the result based on city.

```
SELECT row_number() OVER(PARTITION BY city ORDER BY
player_nr), city, player_nr
FROM tennis_en.player
ORDER BY city;
```

row_number	city	player_nr
1	Den Haag	2
2	Den Haag	6
3	Den Haag	7
4	Den Haag	39
5	Den Haag	67
6	Den Haag	83
7	Den Haag	100
1	Leiden	28
1	Rijswijk	44
1	Rotterdam	192
1	Voorburg	85
1	Zoetermeer	27
2	Zoetermeer	104

• **PARTITION BY city** = reset the row numbering for each city.
• **ORDER BY player_nr inside OVER** = define the order of numbering within each city.
• **ORDER BY city at the end** = control how results are displayed, grouped by city.

Slide 277

277

SELECT INSTRUCTION SEQUENCE

```
SELECT
FROM
WHERE
GROUP BY
HAVING
WINDOW
-- window component
ORDER BY
```

```
SELECT date, player_nr, SUM(amount)
OVER w1, AVG(amount) OVER w2
FROM tennis_en.fine
WINDOW w1 AS (ORDER BY date), w2
AS (ORDER BY player_nr)
ORDER BY date;
```

date	player_nr	sum	avg
2025-03-03	2 Elfring	1.011	253,33333333333
2025-03-03	2 Elfring	1.011	253,33333333333
2025-03-03	6 Ruemmenier	1.011	252,76
2025-03-03	2 Elfring	1.011	253,33333333333
2025-04-12	28 Coole	1.136	226,5714285714
2025-05-23	44 Bakker, de	1.311	234,55555555556
2025-06-12	8 Nieuwenburg	1.961	252,2
2025-06-13	28 Coole	2.111	226,5714285714
2025-06-13	44 Bakker, de	2.111	234,55555555556

Slide 278

278

OVER() VS GROUP BY

- OVER()** supports window-only functions and aggregates-as-window;
- GROUP BY** supports aggregates only.
- Window <> identical to grouping
 - You only have 1 value per group
 - You can have multiple values per window
- Show per player his total number of fines.
 - GROUP BY: returns **one row per player**.
 - WINDOW: returns **each fine row** with the player's total alongside.

Slide 279

279

OVER() VS GROUP BY

- Grouping

```
SELECT player_nr, SUM(amount)
FROM fine
GROUP BY player_nr;
```
- Window

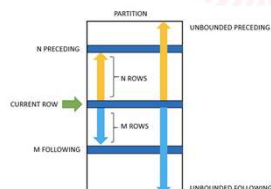
```
SELECT player_nr, SUM(amount) OVER(PARTITION BY player_nr)
FROM fine;
```
- Are these queries equivalent?

Slide 280

280

FRAME

- When you use OVER(...) with an ORDER BY, you can refine the "window frame" using:
 - RANGE** → frame is based on values of the ORDER BY expression(s).
 - ROWS** → frame is based on physical row counts.
 - GROUPS** → frame is based on groups of peer rows with the same ordering value. (less common)



Slide 281

281

FRAME - RANGE

- RANGE** include all rows whose ORDER BY values fall within a specified numeric or temporal range relative to the current row.
- Important: rows with the **same ORDER BY value** are treated as a group — they are either all included or all excluded from the frame.
- Range is the default.

Slide 282

282

EQUAL ORDER

```
SELECT player_nr, SUM(amount)
OVER(ORDER BY player_nr)
```

```
FROM tennis_en.fine
```

```
ORDER BY 1;
```

- Two rows are **equivalent with respect to an ORDER BY** if the database considers them **tied** — i.e., they have identical values for all expressions listed in the ORDER BY clause.
- If rows are equivalent, their **relative order is interchangeable**
- In **window functions**, these are called **peer rows**

player_nr	sum
2	760.00
2	760.00
2	760.00
6	1011.00
8	1261.00
28	1586.00
28	1586.00
44	2111.00
44	2111.00

Slide 283

283

FRAME - ROWS

- A **ROWS** frame counts a **specific number of rows before/after the current row**, based on their physical position in the sorted partition.
- Unlike **RANGE**, rows with the same ORDER BY value are treated **separately**.

- Example:

```
SELECT player_nr, SUM(amount) OVER(ORDER BY
player_nr ROWS BETWEEN UNBOUNDED PRECEDING
AND CURRENT ROW)
FROM tennis_en.fine;
```

player_nr	sum
2	258.00
2	509.00
2	760.00
6	1011.00
8	1261.00
28	1386.00
28	1586.00
44	1936.00
44	2111.00

Slide 284

284

FRAME PARAMETERS

- FRAME parameters:**
 - ROWS/RANGE/GROUPS frame_start
 - ROWS/RANGE/GROUPS BETWEEN frame_start AND frame_end
- frame_start / frame_end options:**
 - UNBOUNDED PRECEDING
 - <N> PRECEDING (e.g. 3 PRECEDING)
 - CURRENT ROW
 - <N> FOLLOWING (e.g. 4 FOLLOWING)
 - UNBOUNDED FOLLOWING

Slide 285

285

RANGE VS ROWS

- In **RANGE** mode, **CURRENT ROW** for a frame_start means that the frame starts with the first row that (according to ORDER BY) is in equal order with the current row, while **CURRENT ROW** for a frame_end means that the frame ends with the last row that (according to ORDER BY) is in equal order.
- In **ROWS** mode, **CURRENT ROW** simply means the current row.

Slide 286

286

RANGE VS ROWS

```
SELECT player_nr, SUM(amount)
OVER(ORDER BY player_nr
RANGE BETWEEN UNBOUNDED
PRECEDING AND CURRENT ROW)
FROM tennis_en.fine;
```

player_nr	sum
2	760.00
2	760.00
2	760.00
6	1011.00
8	1261.00
28	1586.00
28	1586.00
44	2111.00
44	2111.00

```
SELECT player_nr, SUM(amount)
OVER(ORDER BY player_nr
ROWS BETWEEN UNBOUNDED
PRECEDING AND CURRENT ROW)
FROM tennis_en.fine;
```

player_nr	sum
2	258.00
2	509.00
2	760.00
6	1011.00
8	1261.00
28	1386.00
28	1586.00
44	1936.00
44	2111.00

Slide 287

287

RANGE PARAMETERS DEFAULT

- The default frame_clause is:
- RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW**
- With **ORDER BY**, the frame is determined as all rows from the start of the partition to the last row with the same order (according to ORDER BY) as the current row. Without **ORDER BY**, all rows in the partition are used in the window frame because all rows are of the same order as the current row.

Slide 288

288

EXAMPLE WITH LAG

- **LAG()** is a window function that lets you look **backward** to a previous row in the same partition, based on the ordering defined in **OVER()**.
- It retrieves the value of an expression from a row **before the current one**. If no such row exists, it returns NULL.
- Example:

```
SELECT player_nr, amount - lag(amount)
OVER(PARTITION BY player_nr ORDER BY date)
AS evolution
FROM tennis_en.fine;
```

123	player_nr	123	evolution
2	Elfring		[NULL]
2	Elfring		-7
2	Elfring		0
6	Permentier		[NULL]
8	Niewenburg		[NULL]
28	Cools		[NULL]
28	Cools		75
44	Bakker, de		[NULL]
44	Bakker, de		175

Slide 289

289

EXERCISE

- For each team captain, give the team number, player number and the sum of his fines. As the last column, also add the ranking, with the captain with the largest sum in position 1. Sort by rank.

- Example Output:

Team_nr	player_nr	sum	rank
1	6	100	1
2	27	175	2

Slide 290

290

EXERCISE

- What is the sequence of execution?
 - We need team captains only -> team table
 - For each captain we need 4 attributes: team_nr, player_nr, sum of their fines, rank among captains by total fines
 - We need to join fine with team to find the captain
 - We need to aggregate fines per captain
 - We need rank captains by total fines
 - We need to sort the output by rank

```
SELECT
FROM
WHERE
GROUP BY
HAVING
WINDOW
-- window component
ORDER BY
```

Slide 291

291

SOLUTION

```
SELECT team_nr, player_nr, sum(amount),
rank() OVER(ORDER BY sum(amount) DESC)
FROM team LEFT OUTER JOIN fine USING (player_nr)
GROUP BY team_nr, player_nr
ORDER BY 4;
```

- -- penalty sum is calculated per team and captain in this example
- -- so that captain's penalties for that team

Slide 292

292

EXERCISE

- Which cities have players who were fined, how much in total, how many large fines (≥ 200) they accumulated, and how do these cities rank against each other by total fines? Cities with no fines (=0) should not be listed.

Slide 293

293

SOLUTION

```
SELECT
p.city,
SUM(f.amount) AS total_fines,
COUNT(*) FILTER (WHERE f.amount >= 200) AS big_fines,
RANK() OVER (ORDER BY SUM(f.amount) DESC) AS city_rank
FROM tennis_en.player p
LEFT JOIN tennis_en.fine f USING (player_nr)
GROUP BY p.city
HAVING SUM(f.amount) > 0
ORDER BY city_rank;
```

Slide 294

294

NUMBERING FUNCTIONS IN POSTGRES

- **ROW_NUMBER()**
 - Assigns a unique sequential number to each row in the partition, ordered by the ORDER BY clause.
- **RANK()**
 - Assigns a rank starting at 1; ties get the same rank, with **gaps** in numbering.
- **DENSE_RANK()**
 - Like RANK(), but without gaps; ranks are consecutive.
- **NTILE(n)**
 - Divides rows into n buckets of nearly equal size and assigns a bucket number (1..n).

Slide 295

295

AGGREGATE FUNCTIONS IN POSTGRES

All standard aggregates can be used with OVER

- **SUM(expr)** → sum over frame or partition.
- **AVG(expr)** → average.
- **COUNT(expr)** → count of rows.
- **MIN(expr) / MAX(expr)** → min/max value.
- **Statistical aggregates** (VARIANCE, STDDEV, etc.) also work as window functions.

Slide 296

296

VALUE FUNCTIONS IN POSTGRES

- **LAG(value [, offset [, default]])**
 - Returns the value from a row **before** the current one in the partition.
- **LEAD(value [, offset [, default]])**
 - Returns the value from a row **after** the current one in the partition.
- **FIRST_VALUE(value)**
 - Returns the value of the first row in the frame (according to ORDER BY).
- **LAST_VALUE(value)**
 - Returns the value of the last row in the frame.
- **NTH_VALUE(value, n)**
 - Returns the value of the nth row in the frame.

Slide 297

297

DISTRIBUTION FUNCTIONS IN POSTGRES

- **CUME_DIST()**
 - Returns the cumulative distribution (fraction of rows ≤ current row).
- **PERCENT_RANK()**
 - Relative rank of the current row as a fraction between 0 and 1.
- **PERCENTILE_CONT(p)** (ordered-set aggregate, can be windowed)
 - Continuous percentile interpolation.
- **PERCENTILE_DISC(p)** (ordered-set aggregate, can be windowed)
 - Discrete percentile — picks actual row value.

Slide 298

298



OPO: Data Management

Aimée Lynn Backiel
Wim Bertels
Alain Van Der Jeught

Module B: Advanced SQL

Grouping and Filtering

TI / BCS Attribution-NonCommercial-ShareAlike 4.0 Unported Licence 2025-2026

299

SYMBOLS USED IN THIS COURSE

Symbol	Category	Typical Slide Use
	Positioning	Positioning, purpose
	Objective	Objective, goal
	Attention	Warnings, key pitfalls, critical notes
	Theory	Concept explanations, definitions
	Practice	Exercises, hands-on labs
	Clip	Demo, Clip
	Poll	Live questions / Quiz
	Informational	Neutral notes, references, reading tips. Not examinable

Slide 300

300

PURPOSE

- Perform **multidimensional aggregation** efficiently using ROLLUP, CUBE, and GROUPING SETS — eliminating repetitive UNION ALL queries.
- Understand how to compute **hierarchical subtotals** (with ROLLUP), **all possible dimensions** (with CUBE), or **custom combinations** of aggregates (with GROUPING SETS).
- Introduce FILTER to apply **conditional aggregation**, such as computing counts only for specific groups, **within** the same query.

Slide 301

301

OBJECTIVES

- Write queries** that use:
 - ROLLUP to generate subtotals and grand totals.
 - CUBE to generate all combinations of groupings for multidimensional analysis.
 - GROUPING SETS for custom grouping combinations in a single query.
- Explain the difference** between ROLLUP, CUBE, and GROUPING SETS, and choose when to use each.
- Use the FILTER clause** in aggregate functions to apply conditional logic.

Slide 302

302

STANDARD GROUP BY

- Produces one aggregate row **per unique combination** of the specified columns.
- Example


```
SELECT player_nr, SUM(amount) AS total_amount
FROM tennis_en.fine
GROUP BY player_nr;
```
- Limitation:** It only gives you one level of grouping at a time.
- If you want subtotals or multiple grouping levels, you'd need **multiple queries with UNION ALL**.
- Solution => Extended Grouping operators

player_nr	total_amount
44 Bakker, de	525
28 Giele	325
6 Hermans	251
2 Eftink	760
8 Nieuwenburg	250

Slide 303

303

EXTENDED GROUPING OPERATORS – ROLLUP

- ROLLUP** is a **hierarchical aggregation operator**.
- It generates subtotals by progressively removing columns from right to left in the grouping list.
- Always includes the **grand total**.
- Think of it as "drill-down" aggregation.
- Example:

```
SELECT player_nr, SUM(amount) AS total_amount
FROM tennis_en.fine
GROUP BY ROLLUP (player_nr);
```

player_nr	total_amount
NULL	2191
44 Bakker, de	525
28 Giele	325
6 Hermans	251
2 Eftink	760
8 Nieuwenburg	250

Grand total

Slide 304

304

EXTENDED GROUPING OPERATORS – ROLLUP

- ROLLUP with multiple columns**
- Example:


```
SELECT city, player_nr, SUM(amount) AS total_amount
FROM tennis_en.fine
INNER JOIN tennis_en.player USING (player_nr)
GROUP BY ROLLUP (city, player_nr);
```

city	player_nr	total_amount
NULL	NULL	2191
Rijswijk	44 Bakker, de	525
Den Haag	2 Eftink	760
Londen	28 Giele	325
Den Haag	6 Hermans	251
Rijswijk	8 Nieuwenburg	250
Londen	NULL	325
Rijswijk	NULL	775
Den Haag	NULL	1011

Processing :
(city, player_nr) → per city+player totals
(city) → city subtotal (all players in that city)
() → grand total (all cities, all players)

Order in the GROUP BY is important!
Order of rows is **not guaranteed** unless you add ORDER BY

Slide 305

305

EXTENDED GROUPING OPERATORS – ROLLUP

- Without ROLLUP, you'd need to write **multiple GROUP BY queries** and combine them with UNION ALL.
- Example:


```
SELECT city, player_nr, SUM(amount) AS total_amount
FROM tennis_en.fine INNER JOIN tennis_en.player USING (player_nr)
GROUP BY city, player_nr
UNION
SELECT city, NULL, SUM(amount) AS total_amount
FROM tennis_en.fine INNER JOIN tennis_en.player USING (player_nr)
GROUP BY city
UNION
SELECT NULL, NULL, SUM(amount) AS total_amount
FROM tennis_en.fine INNER JOIN tennis_en.player USING (player_nr);
```

Slide 306

306

EXTENDED GROUPING OPERATORS – CUBE

- **CUBE** generates **all possible combinations of the specified columns**.
- Produces a **multidimensional cross-tab** (like pivoting along multiple dimensions).
- Includes the grand total as well.
- Example

```
SELECT city, player_nr, SUM(amount) AS total_amount
FROM tennis_en.fine
INNER JOIN tennis_en.player USING (player_nr)
GROUP BY CUBE (city, player_nr);
```

Slide 307

307

EXTENDED GROUPING OPERATORS – CUBE

A? city	13? player_nr	13? total_amount
[NULL]	[NULL]	2,111
Rijswijk	44 Bakker, de	525
Den Haag	2 Elfring	760
Leiden	28 Cools	325
Den Haag	6 Permentier	251
Rijswijk	8 Nieuwenburg	250
Leiden	[NULL]	325
Rijswijk	[NULL]	775
Den Haag	[NULL]	1,011
[NULL]	44 Bakker, de	525
[NULL]	28 Cools	325
[NULL]	6 Permentier	251
[NULL]	2 Elfring	760
[NULL]	8 Nieuwenburg	250

Processing :
 (city, player_nr) → totals per (city, player)
 (city) → **city subtotals** (all players in that city)
 (player_nr) → **player subtotals** (across all cities)
 () → **grand total** (all players, all cities)

Order of rows is **not guaranteed** unless you add ORDER BY

Slide 308

308

EXTENDED GROUPING OPERATORS – GROUPING SETS

- **GROUPING SETS** lets you **explicitly specify which combinations of columns to group by**.
- It is more flexible: you decide the exact sets of groupings you want, instead of all or hierarchical.
- Example

```
SELECT gender, city, COUNT(*) AS total
FROM tennis_en.player
GROUP BY GROUPING SETS ((city),(gender))
ORDER BY 2, 1
```

A? gender	A? city	13? total
[NULL]	Den Haag	7
[NULL]	Leiden	1
[NULL]	Rijswijk	2
[NULL]	Rotterdam	1
[NULL]	Voorburg	1
[NULL]	Zoetermeer	2
F	[NULL]	5
M	[NULL]	9

Slide 309

309

GROUPING SETS VS. ROLLUP

ROLLUP(c1,c2,c3) == GROUPING SETS (
 (c1, c2, c3),
 (c1, c2),
 (c1),
 ()
)

Slide 310

310

GROUPING SETS VS. CUBE

CUBE(c1,c2,c3) ==
 ALL combinations
 GROUPING SETS (
 (c1,c2,c3),
 (c1,c2),
 (c1,c3),
 (c2,c3),
 (c1),
 (c2),
 (c3),
 ()
)

Slide 311

311

COMBINATIONS

- Multiple groupings are possible together
- Capabilities :
 - 1 grouping set + 1 simple : "addition"
 - 2 or more grouping sets : "multiplication" of specifications similar to cartesian product
 - Multiple grouping sets together : expansion into equivalent grouping sets
 - E.g. GROUP BY GROUPING SETS (E1,E2),E3
 == GROUP BY GROUPING SETS ((E1,E3),(E2,E3))
- Union !

Slide 312

312

COMBINATIONS

GROUP BY a, CUBE (b, c), GROUPING SETS ((d), (e))

Is equivalent to:

```
GROUP BY GROUPING SETS (
  (a, b, c, d), (a, b, c, e),
  (a, b, d), (a, b, e),
  (a, c, d), (a, c, e),
  (a, d), (a, e)
)
```

Slide 313

313

COMBINATIONS

GROUP BY ROLLUP (a, b), ROLLUP (a, c)

```
GROUP BY GROUPING SETS (
  (a, b, c),
  (a, b),
  (a, b),
  (a, c),
  (a),
  (a),
  (a, c),
  (a),
  ()
)
```

When specifying multiple grouping items together, the final set of grouping sets might contain duplicates

Slide 314

314

COMBINATIONS

GROUP BY DISTINCT ROLLUP (a, b), ROLLUP (a, c)

```
GROUP BY GROUPING SETS (
  (a, b, c),
  (a, b),
  (a, c),
  (a),
  ()
)
```

If these duplicates are undesirable, they can be removed using the DISTINCT clause directly on the GROUP BY

This is not the same as using SELECT DISTINCT because the output rows may still contain duplicates. If any of the ungrouped columns contains NULL, it will be indistinguishable from the NULL used when that same column is grouped

• <> SELECT DISTINCT

Slide 315

315

FILTER CLAUSE

- The FILTER clause let you apply a **WHERE-like condition only to a specific aggregate function, within the same query.**
- This means you can compute **multiple different aggregates** from the same dataset, in one scan of the table, instead of writing multiple queries or CASE expressions.
- Example:

```
SELECT player_nr, SUM(amount), SUM(amount) FILTER (WHERE amount>200)
FROM tennis_en.fine
GROUP BY player_nr;
```

player_nr	sum	sum
44 Bakker-de	535	350
28 Coats	325	[NULL]
6 Hermantier	251	251
2 Etring	760	760
8 Neuenburg	250	250

Slide 316

316

ON AGGREGATION

- Limit, filter the transmitted values for aggregation.
- aggregate_name (expression [, ...] [order_by_clause]) [FILTER (WHERE filter_clause)]
- aggregate_name (ALL expression [, ...] [order_by_clause]) [FILTER (WHERE filter_clause)]
- aggregate_name (DISTINCT expression [, ...] [order_by_clause]) [FILTER (WHERE filter_clause)]
- aggregate_name (*) [FILTER (WHERE filter_clause)]

Slide 317

317

EXERCISE

- Provide the player number and number of fines for all players with fines, as well as the number of fines for which the amount is greater than or equal to 200 euros.

Slide 318

318

SOLUTION

- Provide the player number and number of fines for all players with fines, as well as the number of fines for which the amount is greater than or equal to 200 euros.

```
SELECT player_nr, COUNT(*) AS "number of fines",
COUNT(*) FILTER (WHERE amount >= 200) AS "number of fines
>= 200"
FROM tennis_en.fine
GROUP BY player_nr;
```

player_nr	number of fines	number of fines >= 200
44 Bakker, de	2	1
28 Cools	2	1
6 Permentier	1	1
2 Elfring	3	3
8 Nieuwenburg	1	1

319

GROUPING AND FILTERING USE CASES

- Efficient Multidimensional Aggregation**
 - Without them, you'd need to write multiple GROUP BY queries and combine them with UNION ALL.
 - With ROLLUP/CUBE/GROUPING SETS → done in one query.
- Cleaner and More Readable SQL**
 - Keeps complex analytical queries short and easier to understand.
 - Instead of repeating logic in several subqueries, you declare all aggregations once.
- Performance Benefits**
 - Database engine can optimize grouping work internally.
 - Avoids re-scanning the same data multiple times.
 - Especially important for large datasets.

320

GROUPING AND FILTERING USE CASES

- Supports OLAP-Style Analysis**
 - Business intelligence queries often require subtotals, cross-totals, and roll-ups.
 - These operators bring data warehouse-style aggregation directly into SQL.
 - ROLLUP = hierarchical reports (e.g., continent → country → city totals).
 - CUBE = pivot-style multidimensional reports (all dimension combinations).
 - GROUPING SETS = flexible reporting with exactly the totals you need.
- Conditional Aggregation with FILTER**
 - Let's you compute multiple, different aggregates in the same scan.
 - E.g., "Count planets" and "Count moons" in one query.
 - Avoids multiple passes over the data.

321

OPO: Data Management

Aimée Lynn Backiel
Wim Bertels
Alain Van Der Jeught

Module B: Advanced SQL Indexes

TI / BC5 Attribution-NonCommercial-ShareAlike 4.0 Unported Licence 2025-2026

322

SYMBOLS USED IN THIS COURSE

Symbol	Category	Typical Slide Use
	Positioning	Positioning, purpose
	Objective	Objective, goal
	Attention	Warnings, key pitfalls, critical notes
	Theory	Concept explanations, definitions
	Practice	Exercises, hands-on labs
	Clip	Demo, Clip
	Poll	Live questions / Quiz
	Informational	Neutral notes, references, reading tips. Not examinable

323

PURPOSE

- Improve **query performance** by reducing I/O.
- Speed up **row retrieval** using efficient search structures (e.g., B-tree).
- Support **fast filtering, joins, grouping, and sorting**.
- Enforce **uniqueness** (primary keys, unique constraints).
- Optimize **range queries** and **ordered scans**.
- Trade-off: extra **storage** + overhead on **inserts/updates/deletes**.

324

OBJECTIVES

- Explain the **purpose** and role of indexes.
- Distinguish between **clustered, non-clustered, unique, composite, partial** indexes.
- Identify **when to create indexes** (based on queries, selectivity, joins, sorting).
- Recognize the **trade-offs** (performance gain vs storage & maintenance cost).

Slide 325

325

INDEX

- Purpose: improve query performance
- How: by reducing the number of rows/pages that need to be scanned!
- OS:
 - Tables are stored as **files** on disk.
 - A file is divided into **pages (blocks)**.
 - Rows are placed inside these pages.
 - When the database needs a row:
 - It **finds which page** contains the row.
 - The **page is read** from disk (or from memory if cached).
 - The **row is located inside the page** and returned.



Slide 326

326

OPERATION OF AN INDEX

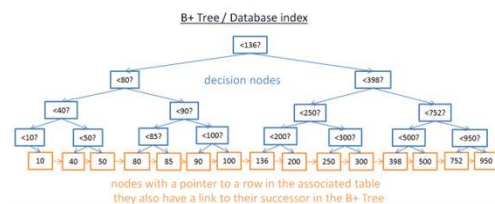
- 2 lookup methods in SQL:
 - **Sequential search method (full table scan):**
 - Read row by row
 - Time-consuming and inefficient
 - **Indexed search method: index (e.g. B-tree)**
 - Tree structure
 - Root & Nodes
 - Leaf pages contain pointers to the data (page+row)
 - 2 methods:
 - Point lookups: searching rows with a given value
 - Range scans: going through the whole table via a sorted column (efficient when using a clustered index)

Slide 327

327

B-TREE (DEFAULT)

- Binary search tree (node can have 2+leaves)
- Best for $> < =$ operators

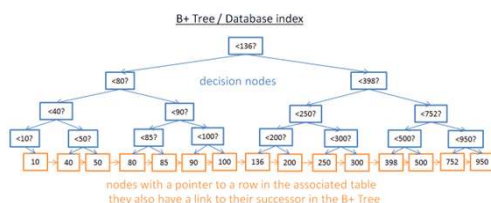


Slide 328

328

B-TREE (EXAMPLE)

- Sequence: max 15 steps
- Via tree: max 4 steps, namely $\log_2 15+$



Slide 329

329

OPERATION OF AN INDEX

- Updates
 - When table is modified: index is modified
- Types & usage
 - Index can be created on unique or non-unique columns
 - A table can have:
 - multiple indexes
 - one clustered index
 - composite index (on multiple columns)
- Caution
 - Index takes up extra storage space
 - Too many indexes reduces write performance
 - Indexes sometimes need rebuilding/reorganizing (especially if fragmented)
 - Multiple index forms are possible: b-tree, hash, GIN,...

Slide 330

330

PLANNER / OPTIMISER

- **Conceptual view:** when we write SQL we may think in loops or nested processing and in a specific order of SELECT processing.
- **Internal view:** The database engine may execute the query in a completely different way (e.g., hash join instead of nested loop, indexes instead of scanning). You do not control the exact steps.
- SQL is a declarative (programming) language, not an imperative (programming) language.
 - You describe what should happen (logically), not (explicitly) how it should happen.
 - The query optimizer chooses the best execution plan.
 - This is one of the strengths of SQL.
- Reason conceptually about the logic. The DBMS will handle the physical details.

Slide 331

331

OPTIMISER

- Seeks the best execution strategy for a query
- The optimiser bases its choice on:
 - Expected processing time
 - Number of rows
 - Indexes
 - Internal statistics
 - Join methods
 - ...
- Goal: minimize cost (CPU, I/O, memory)

Slide 332

332

CREATE INDEX

- No ANSI or ISO specification
 - Vendor specification
- <https://www.postgresql.org/docs/current/sql-createindex.html>

Slide 333

333

CREATE INDEX (POSTGRESQL)

- **Regular index**
 - `CREATE INDEX player_post_code_idx ON player (post_code asc);`
- **Unique composite index**
 - `CREATE UNIQUE INDEX player_name_initials_idx ON player (last_name, initials);` -- Enforces that no two players can have the exact same combination of name + initials.
- **Partial index**
 - `CREATE INDEX player_name_initials_partial_idx ON player (last_name, initials) WHERE player_nr < 100;` -- save space
- **Clustered index**
 - `CLUSTER player USING player_name_initials_idx;` --physically reorders the table rows on disk according to (last_name, initials)

Slide 334

334

DELETE/UPDATE

What is the effect of a clustered index?

Slide 335

335

REINDEX

- Drops the existing index internally and recreates it.
- The physical structure of the index is rebuilt using the current table data.
- Can fix or improve:
 - **Index corruption** (rare, but possible after crashes).
 - **Fragmentation / bloat** (indexes grow inefficient after many inserts, updates, deletes).
 - **Performance degradation** (queries using the index become slower).
- Vendor specification: <https://www.postgresql.org/docs/current/sql-reindex.html>
- Postgresql:
 - `REINDEX INDEX a_index;`
 - `REINDEX TABLE a_table;`
 - `REINDEX DATABASE a_database;`

Slide 336

336

INDEX MANAGEMENT

- **CREATE**
- **ALTER**
 - **ALTER INDEX** big_idx
 - **SET TABLESPACE** somewhere_other;
 - If an index enforces a constraint, drop the **constraint**, not the index.
- **DROP**
- Most SQL products :
 - Automatic index for **PRIMARY KEY** and **UNIQUE** constraints; **foreign keys** are auto-indexed **only in some DBMS**
 - name is derived from the name of the table and the relevant columns

Slide 337

337

WHEN TO INDEX?

- Index:
 - Benefit: index speeds up processing
 - Disadvantage:
 - index takes up storage space
 - every change requires adjustment of index
=> processing slows down

Slide 338

338

WHICH COLUMNS?

- Guidelines for choice of columns :
 - Unique index on candidate keys
 - Index to referenced keys
 - Index on columns on which (many) selections are made
 - Size of table
 - Cardinality (different values) of the table
 - Distribution (sharing) of values
 - Index on a combination of columns
 - Index on columns being sorted
 - ..

Slide 339

339

SPECIAL INDEX FORMS

- Multi-table index :
 - = index on columns in multiple tables
- Virtual column index :
 - = index on an expression
- Selective index
 - = index on a selection of rows
- Hash index
 - = index based on the address of the page
- Bitmapindex
 - = interesting if there are many duplicate values

Slide 340

340

NEWER INDEX FORMS: GIST AND SP-GIST

- GIST (Generalized Search Tree)
 - Balanced tree structure
 - template for different index schemes
 - Supports "clustering" based on arbitrary similarity/distance measures.
 - Good for data types that don't fit standard B-tree orderings, e.g. comparing intervals, GIS, contains, closest neighbours, text
- SP-GIST (space-partitioned GIST)
 - Doesn't need to be balanced
 - No overlap between clusters (GIST)

Slide 341

341

NEWER INDEX FORMS: GIN AND BRIN

- GIN (Generalized Inverted Index)
 - Interesting with many duplicate values
 - Multiple lookup values are created simultaneously (useful for row, text, json,...)
- BRIN : for large clustered tables
 - Small, lower-performing unless:
 - min/max value per block

Slide 342

342

CONCLUSION

An index on every table and column?

Slide 343

343

SUMMARY

- B-tree:
 - good standard, < <= = > >
- Hash:
 - alternative for 1 row, =
- GIN:
 - efficient for doubles, multiple lookup values per field, <@ @> = &&
- GiST:
 - many features, less performant, << &< ... <<| &<| <@ ~= &&
- Sp-GiST:
 - non-overlapping GiST, << >> ~= <@ <<| |>>
- BRIN:
 - large clustered data, < <= = > >

Slide 344

344

INDEX CATALOG_TABLE

- Postgresql: pg_index
- <http://www.postgresql.org/docs/current/static/catalog-pg-index.html>
- <http://www.postgresql.org/docs/current/static/internals.html>
- References: Index Internals, Heikki Linnakangas (Pivotal)

Slide 345

345

OPO: Data Management

Aimée Lynn Backiel
Wim Bertels
Alain Van Der Jeught

Module B: Advanced SQL
Query plan

T1 / BCS Attribution-NonCommercial-ShareAlike 4.0 Unported Licence 2025-2026

346

SYMBOLS USED IN THIS COURSE

Symbol	Category	Typical Slide Use
	Positioning	Positioning, purpose
	Objective	Objective, goal
	Attention	Warnings, key pitfalls, critical notes
	Theory	Concept explanations, definitions
	Practice	Exercises, hands-on labs
	Clip	Demo, Clip
	Poll	Live questions / Quiz
	Informational	Neutral notes, references, reading tips. Not examinable

Slide 347

347

PURPOSE

- SQL queries can often be written in many ways — but **not all are equally efficient**.
- The **PostgreSQL optimizer** decides *how* a query will run (scan method, join type, use of indexes, etc.).
- A **query plan** reveals the **execution strategy** chosen by the database, helping us see why certain queries perform efficiently while others do not.
- We study query plans so we can **design queries and indexes that perform well at scale**.

Slide 348

348

OBJECTIVES

- **Interpret a query plan** (costs, row estimates,...).
- **Differentiate** between EXPLAIN, ANALYZE, and EXPLAIN ANALYZE.
- **Understand why statistics matter** for query performance.
- **Detect misestimates** by comparing planned vs actual execution.
- **Apply EXPLAIN tools** safely (choose EXPLAIN vs EXPLAIN ANALYZE depending on query impact).

Slide 349

349

QUERY PLAN

- A **query plan** (also called an **execution plan**) is the **step-by-step strategy** that a database management system (DBMS) chooses to execute a SQL query.
- It is created by the **query optimizer** after parsing and analysing your SQL.
- It shows the **order of operations** (e.g., which tables to access first, how to apply joins, when to sort or group).
- It describes the **access paths**: full table scan, index scan, etc.
- It estimates the **costs** of different strategies and picks the most efficient one.

Slide 350

350

WHY IS QUERY PLAN IMPORTANT?

- Lets you see *why* a query might be slow.
- Helps in **tuning queries** (adding indexes, rewriting SQL, changing join strategies).
- Execution plans can be viewed in PostgreSQL using commands (no standard ANSI SQL):
 - EXPLAIN
 - EXPLAIN ANALYZE

Slide 351

351

ACCESS PATH

- An **access path** is the method the DBMS chooses to retrieve rows from a table (with or without index) when executing a query.
- DBMS dependent

Slide 352

352

EXPLAIN VS EXPLAIN ANALYZE VS ANALYZE

- **EXPLAIN**
 - Shows the **query execution plan** that the PostgreSQL **planner/optimizer** thinks it will use.
 - Provides **estimated costs**, **row counts**, and **join strategies**.
 - **Does not actually execute** the query.
 - Useful when you want to check the plan quickly without running a potentially slow or dangerous query.
- **ANALYZE**
 - Updates table and column statistics.
 - Provides the planner with accurate information so EXPLAIN estimates are reliable.
 - Can be run on a single table or the entire database, usually done after big data loads or table changes.
- **EXPLAIN ANALYZE**
 - Actually **executes the query** and shows:
 - The **real runtime statistics** (actual time, actual rows processed).
 - Whether the optimizer's **estimates were accurate** or not.
 - Useful for detecting misestimates.
 - Can be expensive, since it runs the query fully.

Slide 353

353

COMMON ACCESS PATHS IN POSTGRES

- **Sequential Scan (Seq Scan)**
 - Reads every row of the table.
 - Chosen when:
 - No suitable index exists, or
 - The optimizer expects most/all rows will be returned.
- **Index Scan**
 - Uses a **B-tree** or **other index** to find matching rows efficiently.
 - Supports:
 - **Index unique scan** (exact match on a unique/primary key).
 - **Index range scan** (conditions like BETWEEN, <, >).
 - Each row requires a **table lookup** (heap access) unless all needed columns are in the index.

Slide 354

354

COMMON ACCESS PATHS IN POSTGRES

- **Index-Only Scan**
 - Reads rows **entirely from the index**, without touching the table.
 - Works if the index covers all queried columns **and** visibility info is available (each row version has metadata telling whether it is **visible** to the current transaction).
- **Bitmap Index Scan**
 - Retrieves multiple matching row pointers from one or more indexes, combines them in a bitmap, then fetches the table rows in bulk.
 - Efficient for many matches (e.g., WHERE col IN (...)).
- **Parallel Scans**
 - Variants of Seq Scan, Index Scan, or Bitmap Scan executed by multiple worker processes in parallel.

Slide 355

355

COST

- In PostgreSQL, the **cost** is a **planner's estimate** of how expensive an operation will be, measured in **arbitrary cost units** (not seconds or milliseconds).
 - It's not actual runtime.
 - It's used by the **query optimizer** to compare different possible access paths and join strategies.
 - The plan with the **lowest total cost** is chosen.

Slide 356

356

COST

- Example:
EXPLAIN
SELECT *
FROM celestial_object;

```
QUERY PLAN
text
Seq Scan on celestial_object (cost=0.00..1.77 rows=77 width=45)
```

- **0.00** → *startup cost* (time before first row can be returned).
- **1.77** → *total cost* (time to process the entire scan).
- **rows=77** → estimated number of rows returned.
- **width=45** → estimated average row size (in bytes).

Slide 357

357

PLANNING/EXECUTION TIME

- **Planning time** is how long the DBMS spent deciding *how* to run the query.
- **Execution time** is the **actual time in milliseconds** it took to **run the query**.
- **Example**
EXPLAIN ANALYZE
SELECT *
FROM celestial_object;

```
QUERY PLAN
text
Seq Scan on celestial_object (cost=0.00..1.77 rows=77 width=45) (actual time=0.026..0.054 rows=73 loops=1)
Planning Time: 0.115 ms
Execution Time: 0.097 ms
```

Slide 358

358

PLANNING/EXECUTION TIME

- **Actual time:**
 - Shown on each operation
 - Format: Actual Time=start..end.
 - start = when the first row was produced.
 - end = when the last row was produced.
 - Helps identify which operations are slow or unexpectedly long.
- **Planning time:**
 - Reported once at the end.
 - Time spent by the **planner/optimizer** to:
 - Parse the query
 - Validate schema
 - Collect statistics
 - Choose the best plan (e.g., Seq Scan vs Index Scan)
- **Execution Time**
 - Reported once at the end.
 - The **total time** to run the query and deliver all rows back to the client.
 - Includes the execution of all operators (like Seq Scan) plus overhead.
 - This is the "real" runtime of your query.

Slide 359

359

COST VS TIME

Aspect	Cost (planner estimate)	Execution/Runtime (real world)
Source	Query optimizer	Query execution engine
Unit	Arbitrary cost units	Milliseconds (ms)
Purpose	Choose cheapest plan	Measure actual performance
Shown by	EXPLAIN	EXPLAIN ANALYZE
Accuracy	Based on statistics (may be wrong)	Actual numbers (ground truth)

The **optimizer** decides between alternative query plans based on **estimated cost**, not **execution time**.
As a DBA/developer, you care about **Execution Time** (real runtime in ms).
But to understand **why** PostgreSQL picked a certain plan, you look at **cost estimates** and compare them to **actual runtime**.

Slide 360

360

EXPLAIN (ANALYZE) IN PGADMIN

```

1 SET SEARCH_PATH TO space_travel;
2 EXPLAIN
3 SELECT *
4 FROM client NATURAL INNER JOIN participation;

```

Data Output Messages Notifications

QUERY PLAN
text

1	Hash Join (cost=1.14..2.31 rows=13 width=134)
2	Hash Cond: (participation.client_nr = client.client_nr)
3	→ Seq Scan on participation (cost=0.00..1.13 rows=13 width=24)
4	→ Hash (cost=1.06..1.06 rows=6 width=122)
5	→ Seq Scan on client (cost=0.00..1.06 rows=6 width=122)

Slide 361

361

EXPLAIN (ANALYZE) IN PGADMIN

```

1 SET SEARCH_PATH TO space_travel;
2
3 SELECT *
4 FROM client NATURAL INNER JOIN participation;

```

Data Output Messages Explain X Notifications

Graphical Analysis Statistics

participation

client

Hash

Hash Inner Join

Statistics per Node Type

Node type	Count	Relation name	Scan count
Hash	1		
Hash Inner Join	1	client	1
Seq Scan	2	participation	1
Seq Scan	1		

Slide 362

362

- Highlight statements
- Press F7 or in toolbar

EXAMPLE WITH 1 TABLE

```

CREATE TABLE one_million
(counter integer,
random_text text);

INSERT INTO one_million
SELECT i, md5(random())::text
FROM generate_series (1, 1000000) AS i;

```

Slide 363

363

EXAMPLE WITH 1 TABLE

```

EXPLAIN
SELECT *
FROM one_million;

```

QUERY PLAN
text

Seq Scan on one_million (cost=0.00..18334.00 rows=1000000 width=37)

Slide 364

364

EXAMPLE WITH 1 TABLE

```

ANALYZE one_million;
EXPLAIN
SELECT *
FROM one_million;

```

QUERY PLAN
text

Seq Scan on one_million (cost=0.00..18334.00 rows=1000000 width=37)

Slide 365

365

EXAMPLE WITH 1 TABLE

```

EXPLAIN ANALYZE
SELECT *
FROM one_million;

```

QUERY PLAN
text

Seq Scan on one_million (cost=0.00..18334.00 rows=1000000 width=37) (actual time=0.015..43.043 rows=1000000 loops=1)

Planning Time: 0.103 ms

Execution Time: 68.017 ms

Slide 366

366

EXAMPLE WITH 1 TABLE

```
EXPLAIN
SELECT *
FROM one_million
WHERE counter > 500;
```

QUERY PLAN

text

Seq Scan on one_million (cost=0.00..20834.00 rows=999445 width=37)

Filter: (counter > 500)

Slide 367

367

EXAMPLE WITH 1 TABLE - WITH AN INDEX

```
CREATE INDEX ON one_million(counter);
EXPLAIN
SELECT *
FROM one_million
WHERE counter > 500;
```

QUERY PLAN

text

Seq Scan on one_million (cost=0.00..20834.00 rows=999420 width=37)

Filter: (counter > 500)

INDEX has no impact?

Slide 368

368

EXAMPLE WITH 1 TABLE - WITH AN INDEX

Let's change the WHERE clause

```
EXPLAIN
SELECT *
FROM one_million
WHERE counter < 500;
```

QUERY PLAN

text

Index Scan using one_million_counter_idx on one_million (cost=0.42..26.56 rows=579 width=37)

Index Cond: (counter < 500)

INDEX has impact?

Slide 369

369

EXAMPLE WITH 1 TABLE - WITH AN INDEX

- Filter (> 500) Seq Scan: no index
- Filter (< 500) Index Scan
- What is the reason?

Slide 370

370

EXAMPLE WITH 1 TABLE - WITH AN INDEX

Force index by disabling sequence scan

```
SET enable_seqscan TO OFF;
EXPLAIN ANALYZE
SELECT *
FROM one_million
WHERE counter > 500;
```

QUERY PLAN

text

Index Scan using one_million_counter_idx on one_million (cost=0.42..36799.28 rows=999420 width=37) (actual time=0.022..82.490 rows=999500 loops=1)

Index Cond: (counter > 500)

Planning Time: 0.200 ms

Execution Time: 104.181 ms

Slide 371

371

EXAMPLE WITH 1 TABLE - WITH AN INDEX

```
SET enable_seqscan TO OFF;
```

QUERY PLAN

text

Index Scan using one_million_counter_idx on one_million (cost=0.42..36799.28 rows=999420 width=37) (actual time=0.022..82.490 rows=999500 loops=1)

Index Cond: (counter > 500)

Planning Time: 0.200 ms

Execution Time: 104.181 ms

```
SET enable_seqscan TO ON;
```

QUERY PLAN

text

Seq Scan on one_million (cost=0.00..20834.00 rows=999420 width=37) (actual time=0.080..61.758 rows=999500 loops=1)

Filter: (counter > 500)

Rows Removed by Filter: 500

Planning Time: 0.214 ms

Execution Time: 86.551 ms

Slide 372

372

EXAMPLE WITH MULTIPLE TABLES

```
CREATE TABLE half_a_million (
  counter integer,
  random_boolean boolean);

INSERT INTO half_a_million
SELECT i, i%2=1
FROM generate_series (1, 500000) AS i;
```

Slide 373

373

EXAMPLE WITH MULTIPLE TABLES

```
EXPLAIN ANALYZE
SELECT *
FROM one_million
INNER JOIN half_a_million
ON one_million.counter = half_a_million.counter ;
```

Slide 374

374

EXAMPLE WITH MULTIPLE TABLES

```
QUERY PLAN
text
Hash Join (cost=15417.00..60081.00 rows=500000 width=42) (actual time=75.706..394.416 rows=500000 loops=1)
  Hash Cond: (one_million.counter = half_a_million.counter)
  -> Seq Scan on one_million (cost=0.00..18334.00 rows=1000000 width=37) (actual time=0.013..26.355 rows=1000000 loops=1)
  -> Hash (cost=7213.00..7213.00 rows=500000 width=5) (actual time=75.653..75.653 rows=500000 loops=1)
    Buckets: 262144 Batches: 4 Memory Usage: 656288
    -> Seq Scan on half_a_million (cost=0.00..7213.00 rows=500000 width=5) (actual time=0.004..21.232 rows=500000 loops=1)
Planning Time: 0.619 ms
Execution Time: 343.454 ms
```

Slide 375

375

EXAMPLE WITH MULTIPLE TABLES WITH INDEX

```
-- Add index
CREATE INDEX ON half_a_million(counter);
EXPLAIN ANALYZE
SELECT *
FROM one_million
INNER JOIN half_a_million
ON one_million.counter = half_a_million.counter ;
```

Slide 376

376

EXAMPLE WITH MULTIPLE TABLES WITH INDEX

```
QUERY PLAN
text
Merge Join (cost=1.03..40013.61 rows=500000 width=42) (actual time=0.055..136.200 rows=500000 loops=1)
  Merge Cond: (one_million.counter = half_a_million.counter)
  -> Index Scan using one_million_counter_idx on one_million (cost=0.42..34317.43 rows=1000000 width=37) (actual time=0.023..37.337 rows=500001 loops=1)
  -> Index Scan using half_a_million_counter_idx on half_a_million (cost=0.42..15212.42 rows=500000 width=5) (actual time=0.020..35.265 rows=500000 loops=1)
Planning Time: 3.492 ms
Execution Time: 145.897 ms

Versus without index

Hash Join (cost=15417.00..60081.00 rows=500000 width=42)
```

Slide 377

377

EXAMPLE WITH AGGREGATIONS

```
EXPLAIN
SELECT COUNT(*)
FROM one_million;
```

```
QUERY PLAN
text
Finalize Aggregate (cost=14542.55..14542.56 rows=1 width=8)
-> Gather (cost=14542.33..14542.54 rows=2 width=8)
  Workers Planned: 2
  -> Partial Aggregate (cost=13542.33..13542.34 rows=1 width=8)
    -> Parallel Seq Scan on one_million (cost=0.00..12500.67 rows=416667 width=0)
```

Slide 378

378

EXAMPLE WITH AGGREGATIONS

```
EXPLAIN ANALYZE
SELECT MAX(random_text)
FROM one_million;
```

QUERY PLAN
text
Finalize Aggregate (cost=14542.55..14542.56 rows=1 width=32) (actual time=59.525..60.562 rows=1 loops=1)
→ Gather (cost=14542.33..14542.54 rows=2 width=32) (actual time=59.451..60.547 rows=3 loops=1)
Workers Planned: 2
Workers Launched: 2
→ Partial Aggregate (cost=13542.33..13542.34 rows=1 width=32) (actual time=52.531..52.532 rows=1 loops=3)
→ Parallel Seq Scan on one_million (cost=0.00..12500.67 rows=416667 width=33) (actual time=0.010..20.897 rows=333333 loops=3)
Planning Time: 0.155 ms
Execution Time: 60.589 ms

Slide 379

379

EXAMPLE WITH AGGREGATIONS

```
-- Add index
CREATE INDEX ON one_million(random_text);
EXPLAIN ANALYZE
SELECT MAX(random_text)
FROM one_million;
```

QUERY PLAN
text
Result (cost=0.47..0.48 rows=1 width=32) (actual time=0.229..0.230 rows=1 loops=1)
InitPlan 1
→ Limit (cost=0.42..0.47 rows=1 width=33) (actual time=0.223..0.224 rows=1 loops=1)
→ Index Only Scan Backward using one_million_random_text_idx on one_million (cost=0.42..46340.43 rows=1000000 width=33) (actual time=0.222..0.222 rows=1 loops=1)
Index Cond: (random_text IS NOT NULL)
Heap Fetches: 0
Planning Time: 0.872 ms
Execution Time: 0.259 ms

Slide 380

380

EXAMPLE WITH GROUPS

```
-- Remove index
DROP INDEX one_million_random_text_idx;
EXPLAIN ANALYZE
SELECT random_text, COUNT(*)
FROM one_million
GROUP BY random_text;
```

QUERY PLAN
text
HashAggregate (cost=105834.00..131459.00 rows=1000000 width=41) (actual time=228.958..426.113 rows=1000000 loops=1)
Group Key: random_text
Planned Partitions: 16 Batches: 81 Memory Usage: 8337kB Disk Usage: 63512kB
→ Seq Scan on one_million (cost=0.00..18334.00 rows=1000000 width=33) (actual time=0.015..30.504 rows=1000000 loops=1)
Planning Time: 0.382 ms
Execution Time: 450.873 ms

Slide 381

381

EXAMPLE WITH GROUPS WITH INDEX

```
-- Add index
CREATE INDEX ON one_million(random_text);
EXPLAIN ANALYZE
SELECT random_text, COUNT(*)
FROM one_million
GROUP BY random_text;
```

QUERY PLAN
text
GroupAggregate (cost=0.42..58840.43 rows=1000000 width=41) (actual time=0.361..189.709 rows=1000000 loops=1)
Group Key: random_text
→ Index Only Scan using one_million_random_text_idx on one_million (cost=0.42..43840.43 rows=1000000 width=33) (actual time=0.342..79.919 rows=1000000 loops=1)
Heap Fetches: 0
Planning Time: 0.864 ms
Execution Time: 208.125 ms

Slide 382

382

COMPARING EQUIVALENT QUERIES

Question: Which celestial objects have been visited at least once?

```
EXPLAIN
SELECT c.objectname
FROM celestial_object c
WHERE c.objectname IN
(SELECT DISTINCT v.objectname
FROM visit v);
```

QUERY PLAN
text
Hash Join (cost=1.33..3.27 rows=6 width=7)
Hash Cond: ((c.objectname).text = (v.objectname).text)
→ Seq Scan on celestial_object c (cost=0.00..1.73 rows=73 width=7)
→ Hash (cost=1.26..1.26 rows=6 width=5)
→ HashAggregate (cost=1.20..1.26 rows=6 width=5)
Group Key: v.objectname
→ Seq Scan on visit v (cost=0.00..1.16 rows=16 width=5)

```
EXPLAIN
SELECT DISTINCT c.objectname
FROM celestial_object c
INNER JOIN visit v
ON c.objectname = v.objectname;
```

QUERY PLAN
text
HashAggregate (cost=3.88..4.04 rows=16 width=7)
Group Key: c.objectname
→ Hash Join (cost=2.64..3.84 rows=16 width=7)
Hash Cond: ((v.objectname).text = (c.objectname).text)
→ Seq Scan on visit v (cost=0.00..1.16 rows=16 width=5)
→ Hash (cost=1.73..1.73 rows=73 width=7)
→ Seq Scan on celestial_object c (cost=0.00..1.73 rows=73 width=7)

Are these queries equivalent?
Which query has the best performance (same starting conditions e.g. cache cleansed, stats accurate), and why?

Slide 383

383

CONCLUSION

- EXPLAIN**
 - Shows the **estimated plan** the optimizer will use.
 - Relies on **statistics** → may be inaccurate if statistics are outdated.
 - Works for **SELECT, INSERT, UPDATE, DELETE** (any DML).
 - Focus first on **total cost** (relative measure of plan "expense").
 - Then check **access paths** (Seq Scan, Index Scan, Bitmap Scan, etc.).
- ANALYZE (command)**
 - Updates table statistics.
 - Solution when EXPLAIN gives poor/misleading row estimates.
- EXPLAIN ANALYZE**
 - Runs the query and shows **actual execution time + actual row counts**.
 - Lets you compare **estimated vs actual** to detect misestimates.

Slide 384

384

DEVELOPER APPROACH

- Write search argumentable (sargable) query for needed columns only.
- Ensure matching (composite) indexes.
- EXPLAIN → sanity-check access paths & costs.
- EXPLAIN (ANALYZE) → compare estimates vs actuals.
- If off: ANALYZE, adjust indexes/predicates, retest.

Slide 385

385



OPO: Data Management

Aimée Lynn Backiel
Wim Bertels
Alain Van Der Jeught

Module B: Advanced SQL Performance Optimisation Guidelines

TI / BCS

Attribution-NonCommercial-ShareAlike 4.0 Unported Licence

2025-2026

386

SYMBOLS USED IN THIS COURSE

Symbol	Category	Typical Slide Use
	Positioning	Positioning, purpose
	Objective	Objective, goal
	Attention	Warnings, key pitfalls, critical notes
	Theory	Concept explanations, definitions
	Practice	Exercises, hands-on labs
	Clip	Demo, Clip
	Poll	Live questions / Quiz
	Informational	Neutral notes, references, reading tips. Not examinable

Slide 387

387

PURPOSE

- Ensure queries and database operations run **efficiently**.
- Minimize **resource usage** (CPU, memory, I/O).
- Improve **response times** for end-users.
- Provide a foundation for **scalable and maintainable** systems.

Slide 388

388

OBJECTIVES

- Identify common **factors affecting SQL performance** (indexes, joins, query structure, operators, etc.).
- Apply **best practices** for writing efficient queries.

Slide 389

389

INTRODUCTION

- Optimiser - Best execution plan?
- Indexes - Increasing query speed?
- Insight - What is happening internally?
- But sometimes not optimal
- => Reformulation for an efficient processing strategy !

Slide 390

390

AVOID THE OR OPERATOR

- OR : index is usually not used
- Alternative (if possible) :
 - Replaced by a condition with IN
 - Replaced by 2 selects with UNION
- Example:
...
WHERE player number = 15
OR player number = 29
OR player number = 55
Solution: where player number in (15, 29, 55)

Slide 391

391

UNNECESSARY TO USE INDEX WITH UNION

- When using UNION, the database may **scan the same table multiple times** — once for each SELECT.
- This can cause **redundant index lookups** (and extra I/O).
- Alternative (if possible) :
 - **Combine all conditions into a single SELECT with OR conditions** instead of multiple UNIONS.

Slide 392

392

AVOID THE NOT OPERATOR

- NOT : index is not used
- Alternative (if possible) :
 - Replace NOT with comparison operators
- Example
...
WHERE NOT (year > 1980)
- Solution: WHERE year <= 1980

Slide 393

393

ISOLATE COLUMNS IN CONDITIONS

- Column in a calculation or scalar function : index is not used
- Alternative (if possible) :
 - Isolate the column
- Example:
...
WHERE admission_year + 10 = 1990
- Solution: WHERE year = 1980

Slide 394

394

USE THE BETWEEN OPERATOR

- AND : usually does not use the index
- Example:
...
WHERE admission_year >= 1985
AND admission_year <= 1990
- Solution: WHERE admission_year BETWEEN 1985 AND 1990

Slide 395

395

CERTAIN FORMS OF LIKE OPERATOR

- LIKE : index is not used if pattern starts with % or _
- Alternative :
 - None, unless.
- Example:
...
WHERE name LIKE '%sen'
- Solution: ???

Slide 396

396

REDUNDANT CONDITIONS WHEN USING JOINS

- Add redundant conditions to force SQL to choose a particular path
- Example

```
...
WHERE fine.player_nr = player.player_nr
AND fine.player_nr = 44
```
- Solution: WHERE fine.player_nr = player.player_nr
AND fine.player_nr = 44
AND player.player_nr = 44
- Both fines **and** players can apply an index lookup on player_nr = 44, which improves performance.



Slide 397

397

AVOID THE HAVING COMPONENT

- Conditions in HAVING : index is not used
- Alternative (if possible)
 - As many conditions as possible in WHERE
- Example --does this belong in the having?

```
...
GROUP BY player_nr
HAVING player_nr >= 40
```
- Solution: WHERE player_nr >= 40
GROUP BY player_nr



Slide 398

398

SELECT COMPONENT: KEEP COMPACT

- SELECT component as compact as possible
 - Omit unnecessary columns from SELECT and avoid *
 - For correlated subquery with exists : one expression consisting of one constant
- Example:

```
SELECT player_nr, last_name
FROM player
WHERE EXISTS (SELECT 1
              FROM fine
              WHERE fine.player_nr = player.player_nr)
```



Slide 399

399

AVOID DISTINCT

- DISTINCT : processing time extended
- Alternative (if possible)
 - Avoid if unnecessary
- Example:

```
SELECT DISTINCT match_nr, last_name
FROM match, player
WHERE match.player_nr = player.player_nr
```
- Solution: SELECT match_nr, last_name



Slide 400

400

ALL OPTION AT SET OPERATORS

- Without ALL : processing time extended
- Data must be sorted to remove duplicates
- Example:

```
SELECT last_name, initials
FROM player
WHERE player_nr = 10
UNION ALL
SELECT last_name, initials
FROM plays
WHERE player_nr = 18
```



Slide 401

401

CHOOSE OUTER-JOINS OVER UNION

- UNION: processing time extended
- Alternative (if possible)
 - Outer-join is better



Slide 402

402

CHOOSE OUTER-JOINS OVER UNION EXAMPLE

```
SELECT player.player_nr, last_name, amount
FROM player, fine
WHERE player.player_nr = fine.player_nr
UNION
SELECT player_nr, last_name, null
FROM player
WHERE player_nr NOT IN (SELECT player_nr FROM fine)
ORDER BY 1;
```



Slide 403

403

CHOOSE OUTER-JOINS OVER UNION SOLUTION

```
SELECT player_nr, last_name, amount
FROM player
LEFT OUTER JOIN fine USING(player_nr)
ORDER BY 1;
```



Slide 404

404

AVOID DATA TYPE CONVERSIONS

- Converting data types : processing time extended
- Alternative (if possible) :
 - Avoiding data type conversion
- Example:

```
SELECT *
FROM player
WHERE player_nr = '15';
```
- Solution: WHERE player_nr = 15



Slide 405

405

ORDER TABLES

- Order of tables can be important
- Depending on the accuracy of internal statistics
- Example:

```
SELECT player.player_nr, last_name, team_nr
FROM player, team
WHERE player.player_nr = team.player_nr;
<=>
SELECT player.player_nr, last_name, team_nr
FROM team, player
WHERE player.player_nr = team.player_nr;
```



Slide 406

406

AVOID ANY AND ALL OPERATORS

- ANY and ALL : index is not used
- Alternative (if possible)
- Example:

```
SELECT player_nr, last_name, birthdate
FROM player
WHERE birthdate <= ALL (SELECT birthdate FROM
player);
```



Slide 407

407

AVOID ANY AND ALL OPERATORS

- Solution:

```
SELECT player_nr, last_name, birthdate
FROM player
WHERE birthdate = (SELECT MIN(birthdate)
FROM player);
```
- Replace with min or max



Slide 408

408

HOW TO CHECK?

- Use
 - Explain
 - Explain analyze



Slide 409